

An Infosys Virtual Internship Report On

**VisionExtract: Isolation from Images using
Image Segmentation**

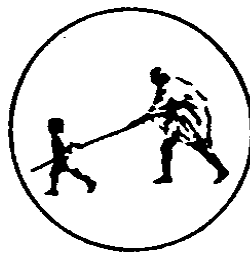
BY

Mr. Shubham Ajay Kunturwar

Under the Guidance

of

Mr. Shivprasad Titare



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Mahatma Gandhi Mission's College of Engineering, Nanded (M.S.)

Academic Year 2025-26

An Infosys Virtual Internship Report On
“VisionExtract: Isolation from Images using Image Segmentation”

Submitted to
DR. BABASAHEB AMBEDKAR TECHNOLOGICAL
UNIVERSITY, LONERE

in partial fulfillment of the requirement for the degree of
BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE & ENGINEERING

By
Mr. Shubham Ajay Kunturwar

Under the Guidance
of

Mr. Shivprasad Titare

(Department of Computer Science and Engineering)



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
MAHATMA GANDHI MISSION'S COLLEGE OF ENGINEERING
NANDED (M.S.)

Academic Year 2025-26

Certificate



This is to certify that the project entitled

“VisionExtract: Isolation from Images using Image Segmentation”

*being submitted by **Mr. Shubham Ajay Kunturwar** to the Dr. Babasaheb Ambedkar Technological University, Lonere , for the award of the degree of Bachelor of Technology in Computer Science and Engineering, is a record of bonafide work carried out by him under my supervision and guidance. The matter contained in this report has not been submitted to any other university or institute for the award of any degree.*

Mr. Shivprasad Titare
Project Guide

Dr. A. M. Rajurkar

H.O.D

Computer Science & Engineering

Dr. G. S. Lathkar

Director

MGM's College of Engg., Nanded

INTERNSHIP OFFER LETTER



Welcome to the Infosys Internship Program | Project Kick-off Session

1 message

Springboard <springboardmentor27300a@gmail.com>
Bcc: s22_modi_satyam@mngmcen.ac.in

Wed, 4 Feb, 2026 at 7:35 am

I am pleased to introduce myself as your mentor, Harshitha, an experienced Data Scientist. Welcome to the Infosys Internship Program, and congratulations on being selected. I look forward to working closely with you as we begin our journey on the project.

Kick-off Session

Our first virtual meeting is scheduled as follows:

Project Name: AirFly Insights: Data Visualization and Analysis of Airline Operations

Date: 05-02-2026

Time: 6:45 - 7:45

Meeting Link: <https://meet.google.com/uig-xafz-bxg>

Program Structure and Expectations

Please take note of the following important points:

1. From DATE onwards, we will have daily sessions within the same time window. The exact timings can be discussed and finalized during the initial session, keeping everyone's feasibility in mind.
2. These sessions are not classroom-style lectures. They are intended for collaboration, guidance, and hands-on problem-solving, so there is no expectation of formal lectures.
3. The first few sessions will focus on:
 - 3.1. Introductions and alignment
 - 3.2. Understanding project objectives and deliverables
 - 3.3. Identifying any knowledge gaps
 - 3.4. Setting clear expectations for the internship
4. Attendance and Communication
 - 4.1. If you need to take leave at any point, please inform me in advance so that LEAVE can be marked in the attendance register. Otherwise, ABSENT will be recorded.
 - 4.2. For all official communication, please use email only to ensure clarity and proper documentation.
5. Professional Conduct
 - 5.1 All interactions are expected to maintain a high standard of professional decorum at all times.
 - 5.2. Respect for peers, mentor guidance, and the overall program structure is mandatory.

We hope this internship will help you develop strong problem-solving instincts and practical skills in data science through real-world project exposure.

I look forward to meeting all of you and starting this collaboration.

Warm regards,

Harshitha

Mentor – Infosys Internship Program

 **DV AirFly Insights.pdf**
237 KB

INTERNSHIP COMPLETION LETTER



Congratulations on successfully completing the Infosys Springboard Internship 6.0

1 message

Infosys Springboard-Support <Springboard-support@infosys.com>

Wed, 29 Apr, 2026 at 3:26 pm



Dear Intern,

Congratulations! You have successfully completed Infosys Springboard Internship 6.0! We're proud of your achievement and the dedication you've shown throughout the internship.

Your certificate will be sent to your registered email address shortly. Keep up the great work and continue striving for excellence!

Announce your achievement and about your internship experience on social media. Tag us @InfosysSpringboard when you do.

Note: Your project is your asset. Refrain from sharing your artifacts in online forums/domains to safeguard your work's ownership

Regards,
Team Infosys Springboard

Keep your learning journey going. Scan the QR code to download the **Infosys Springboard App** and access your courses on the go!



App Store



Play Store

Follow us on Facebook | Instagram | LinkedIn | X (Twitter) | Threads

In case of queries, write to springboard-support@infosys.com.

ACKNOWLEDGEMENT

I would like to express my deepest gratitude to my project guide, **Mr. Shivprasad Titare**, for his invaluable support, guidance, and encouragement throughout this project. His profound knowledge and expertise have been instrumental in the successful completion of this work. His patience and willingness to assist me at every step have greatly enriched my learning experience. His constructive feedback and insightful suggestions have not only helped me overcome challenges but also motivated me to strive for excellence.

I would also like to extend my sincere gratitude to Infosys Springboard for providing me with the opportunity to undertake the Infosys Springboard Virtual Internship 6.0 and gain valuable industry experience. I am deeply thankful to my internship mentor, Ms. G.Srivarsha whose expert guidance, constant support, and practical insights greatly contributed to my professional growth and helped me bridge the gap between academic learning and real-world application. The experience and knowledge gained during this internship have been an integral part of this project's development.

I gladly take this opportunity to thank **Dr. A. M. Rajurkar** (Head of Computer Science & Engineering, MGM's College of Engineering, Nanded). I am heartily thankful to **Dr. G. S. Lathkar** (Director, MGM's College of Engineering, Nanded) for providing facilities during the progress of the project and also for her kind help, guidance and inspiration. Last but not least, I am also thankful to all those who helped, directly or indirectly, develop this project and complete it successfully.

With Deep Reverence,

Shubham Ajay Kunturwar

[B.Tech CSE-A]

ABSTRACT

The rapid growth of digital documents and image-based data has increased the need for intelligent systems capable of automatically extracting meaningful information from visual content. Manual extraction and processing of information from images and scanned documents is time-consuming, error-prone, and inefficient, especially when handling large volumes of data. To address this challenge, this project presents **Vision Extract**, an AI-powered image segmentation and extraction pipeline designed to automate the process of identifying, segmenting, and processing visual information from images and documents.

The proposed system utilizes deep learning and computer vision techniques to perform accurate image segmentation and object extraction. The core of the system is based on the **U-Net Convolutional Neural Network (CNN)** architecture, which is widely recognized for its effectiveness in image segmentation tasks. The model is trained using the **COCO 2017 dataset**, enabling it to learn complex visual patterns and accurately distinguish foreground objects from the background. The system processes uploaded images through a structured pipeline that includes image preprocessing, feature extraction, segmentation mask generation, and final output rendering.

The uploaded image is converted into a matrix representation and passed through an image encoder and segmentation model. The model generates segmentation masks that isolate the required object or region from the background. The extracted output is then returned to the user as a processed image. The system also incorporates preprocessing techniques such as resizing, normalization, and image enhancement to improve segmentation accuracy and overall performance.

To evaluate the effectiveness of the proposed model, various performance metrics such as **Intersection over Union (IoU)**, **Dice Score**, and **Pixel Accuracy** are used. The Vision Extract pipeline can be applied in several real-world domains, including document analysis, medical imaging, object detection, content extraction, autonomous systems, and image editing applications. By combining machine learning, image segmentation, and cloud-based processing, the proposed system provides an efficient, scalable, and intelligent solution for automated visual information extraction.

TABLE OF CONTENTS

TOPICS	PAGE NO.
INTERNSHIP OFFER LETTER	I
INTERNSHIP COMPLETION LETTER	II
ACKNOWLEDGEMENT	III
ABSTRACT	IV
TABLE OF CONTENTS	V-VII
LIST OF FIGURES	VIII
LIST OF TABLES	IX
Chapter 1. INTRODUCTION	1
1.1 Overview of the Project	1
1.2 Internship Overview – Infosys Springboard	1
1.3 Problem Statement	2
1.4 Objectives of the Project	3
1.5 Scope of the Project	3
1.6 Significance of the Study	4
1.7 Use Cases and Applications	4
1.8 Dataset Description – COCO 2017	5
1.9 Existing Technologies and Solutions	6
Chapter 2. LITERATURE REVIEW	7
2.1 Introduction to Image Segmentation	7
2.2 Traditional Image Segmentation Techniques	7
2.3 Semantic Segmentation	9
2.4 Deep Learning in Computer Vision	10
2.5 Convolutional Neural Networks (CNNs)	10
2.6 Encoder–Decoder Architectures	12
2.7 U-Net Architecture	13
2.8 LinkNet and DeepLabV3+	14
2.9 Uniqueness of the Proposed System	15
Chapter 3. DEVELOPMENT METHODOLOGY	17

3.1 System Architecture Overview	17
3.2 Data Collection and Dataset Preparation	18
3.3 Binary Mask Generation	19
3.4 Data Preprocessing Pipeline	20
3.5 Data Augmentation Techniques	21
3.6 Custom PyTorch Dataset Class	21
3.7 Custom U-Net CNN Architecture	22
3.8 Model Selection and Training	23
3.9 Loss Functions and Optimizers	24
3.10 Evaluation Metrics (IoU and Dice Score)	24
Chapter 4. TECHNOLOGY STACK	27
4.1 Programming Language – Python	27
4.2 CPU vs GPU Utilization	28
4.3 Deep Learning Framework – PyTorch	29
4.4 OpenCV for Real-Time Segmentation	30
4.5 Albumentations and Data Augmentation	32
4.6 Segmentation Models PyTorch Library	34
4.7 Flask Framework for Deployment	35
4.8 CUDA Runtime Support	36
4.9 Google Colab Development Environment	37
4.10 Git and Version Control	39
Chapter 5. SYSTEM ARCHITECTURE AND WORKFLOW	41
5.1 Introduction	41
5.2 Overall System Architecture	41
5.3 Image Input and Preprocessing Workflow	42
5.4 Foreground Mask Generation Module	42
5.5 Segmentation Prediction Pipeline	43
5.6 Real-Time OpenCV Segmentation Module	43
5.7 Post-Processing and Morphological Operations	44
5.8 Flask Deployment Workflow	44
5.9 Model Scalability and Optimization	45
5.10 Complete Workflow Analysis	46

Chapter 6. IMPLEMENTATION	47
6.1 Introduction	47
6.2 Implementation Tools and Environment	47
6.3 Dataset Construction and Processing	49
6.4 U-Net Model Training	50
6.5 LinkNet and DeepLabV3+ Training	51
6.6 Evaluation and Validation Results	52
6.7 IoU and Dice Score Analysis	53
6.8 Real-Time Segmentation Results	54
6.9 Prediction Visualization and Output Analysis	55
6.10 Advantages and Limitations	56
FUTURE SCOPE AND ENHANCEMENTS	58
CONCLUSION	61
REFERENCES	62

LIST OF FIGURES

Figure		Page
No.	Name of Figure	No.
1.1	Company Logo	2
1.2	Application Scenarios	5
2.1	Raw Pixels To Labelled Regions	7
2.2	Symmetric U-Net Architecture with Skip Connections	13
2.3	DeepLabV3+ Architecture	15
3.1	System Architecture	18
3.2	Evaluation and Result	25
4.1	Real Time Segmentation web App	31
6.1	Training Loss Convergence	51
6.2	User Interface Showing the Foreground Extraction Result	55
6.3	Qualitative Analysis of Model Predictions	56

LIST OF TABLES

Table No.	Name of Table	Page No.
2.1	Characteristics and Limitations of Different Algorithm	11
6.1	Hardware and Software Environment Specifications	48
6.2	Quantitative Evaluation Metrics of Trained Architectures	53

INTRODUCTION

1.1 Overview of the Project

In the rapidly evolving domain of artificial intelligence and digital media processing, the ability to automatically and accurately analyze the contents of an image is of paramount importance. Among the various tasks within computer vision, extracting a specific subject from its surrounding environment stands out as both a fundamental challenge and a highly sought after capability. The "Vision Extract" project is engineered to address this exact requirement. It is a comprehensive machine learning initiative focused on building, training, and deploying a deep neural network capable of performing high fidelity semantic segmentation to isolate the primary subject within an arbitrary digital photograph.

The overarching objective of the Vision Extract system is to replicate the manual "cutout" or masking functionality traditionally performed by graphic designers, but at a fraction of the time and with scalable consistency. Given an input image containing a subject against a potentially cluttered or complex background, the proposed model processes the visual data through advanced convolutional architectures to generate a pixel precise binary mask. This mask is then algorithmically applied to the original image, resulting in a synthesized output where the main subject retains its original color space and high resolution details, while every pixel classified as the background is uniformly rendered black. This end to end pipeline not only demonstrates the theoretical application of deep learning in spatial analysis but also culminates in a practical, deployable web application.

1.2 Internship Overview – Infosys Springboard

This project was conceptualized and executed as the capstone deliverable for the Infosys Springboard internship program. Infosys Springboard is a premier, digital first learning and internship platform designed to bridge the gap between academic theoretical knowledge and the practical, rigorous demands of the modern software engineering and IT services industry. The program is structured to provide emerging professionals with an immersive corporate experience, focusing on next generation technologies such as Artificial Intelligence, Machine Learning, Cloud Computing, and Full Stack Development.

During the internship, participants are introduced to industry standard development methodologies, including Agile project management, continuous integration, and robust version control. The curriculum requires interns to navigate through a series of structured milestones from initial dataset acquisition and exploratory data analysis to model architecture design, hyperparameter tuning, and eventual deployment via REST APIs. By operating within this structured, deadline driven environment, the internship ensures that the resulting Vision Extract project is not merely an academic exercise contained within a Jupyter Notebook, but a fully realized software solution that meets professional standards of code quality, documentation, and architectural scalability.



Fig 1.1: Company Logo

1.3 Problem Statement

In contemporary media workflows, isolating a foreground subject from its background is a ubiquitous requirement. However, achieving pixel perfect isolation is notoriously difficult due to several complicating factors: dynamic and uneven lighting conditions, subjects that share color palettes with their backgrounds (color camouflage), intricate boundary details like hair or transparent fabrics, and heterogeneous background clutter. Historically, this task has required intensive manual intervention using tools like the magnetic lasso or magic wand in photo editing software. This manual approach is highly unscalable, labor intensive, and prone to human error, making it entirely unsuitable for bulk image processing, real time video analysis, or automated e commerce cataloging.

Therefore, the formal problem statement for this project is defined as follows: *To architect, train, and deploy a robust machine learning model capable of automatically extracting the main subject from an arbitrary, diverse set of input images without requiring human in the loop intervention.* The system must accurately predict a binary segmentation

mask that isolates the subject, allowing the background to be programmatically replaced with a uniform black canvas, regardless of the input image's native complexity.

1.4 Objectives of the Project

To systematically address the challenges outlined in the problem statement, the Vision Extract project was divided into a series of specific, actionable objectives. These objectives span the entire machine learning lifecycle:

- **Algorithm Comprehension:** Understand and apply the principles of semantic segmentation using deep convolutional neural networks tailored for pixel wise classification tasks.
- **Data Pipeline Development:** Implement rigorous data preprocessing strategies, specifically focusing on converting multi class mask annotations into binary formats, handling image normalization, and applying data augmentation to prevent model overfitting.
- **Model Construction and Training:** Build, train, and validate an image segmentation model architecture capable of accurately distinguishing foreground subjects from diverse backgrounds.
- **Performance Evaluation:** Evaluate the model's accuracy using standard quantitative metrics, including Intersection over Union (IoU), Dice coefficient, and pixel wise accuracy.
- **System Deployment:** Deploy the trained model into a production like environment by building a Backend REST API that serves a User Interface, allowing users to upload images and receive the isolated subject as an HTTP response.
- **Documentation:** Prepare detailed technical documentation and presentations detailing the methodology, challenges, and results of the internship project.

1.5 Scope of the Project

The scope of this project encompasses the complete, end to end development of a machine learning feature, defining the precise boundaries of the technical implementation.

In Scope: The project includes the acquisition and preparation of the COCO 2017 dataset, focusing specifically on establishing image to mask resolution consistency and binary mask conversion. It covers the architectural design of a segmentation network utilizing an encoder decoder structure. Furthermore, the scope includes the development of a basic web

application where a frontend UI communicates via HTTP requests (sending images as Blobs) to a Python based Backend REST API, which runs the inference and returns the black background image.

Out of Scope: The project strictly focuses on static image processing. Real time video frame segmentation, deployment to edge or mobile devices (iOS/Android native apps), and the classification of background elements (panoptic segmentation) are explicitly excluded from the current project scope to ensure feasibility within the internship timeline.

1.6 Significance of the Study

The significance of the Vision Extract project lies in its contribution to automating visually intensive, repetitive tasks that traditionally require human expertise. By successfully training a model to understand the semantic boundaries of a subject, this study demonstrates how deep learning can drastically reduce operational bottlenecks in digital workflows. From an academic and engineering perspective, this project serves as a comprehensive case study in handling large scale, complex datasets like COCO 2017, proving the efficacy of encoder decoder architectures in retaining high resolution spatial data. Furthermore, by taking the model out of the experimental phase and encapsulating it within a REST API, the study highlights the critical intersection of Data Science and Software Engineering (MLOps), showcasing how AI models must be packaged to be truly useful in real world, scalable cloud environments.

1.7 Use Cases and Applications

The capability to automatically isolate a main subject has vast commercial and creative applications. The Vision Extract model directly addresses the following use cases:

- **E Commerce and Retail:** Online retailers require product images to have uniform, clean backgrounds (usually white or transparent). Automated subject extraction allows platforms to process bulk uploads from various vendors, standardizing the catalog without manual graphic design work.
- **Virtual Conferencing:** Applications like Zoom and Microsoft Teams utilize real time segmentation to differentiate the human user from their physical room. The principles developed in this project are the foundation for features that blur backgrounds or replace them with virtual environments to maintain user privacy.
- **Photography and Digital Art:** Professional photo editing software integrates these

models to act as intelligent selection tools, allowing artists to instantly isolate subjects for compositing, color grading, or applying localized visual effects.

- **Augmented Reality (AR):** AR systems require a deep understanding of spatial boundaries to seamlessly integrate digital objects behind or in front of real world subjects, necessitating highly accurate, pixel wise segmentation masks.



Fig 1.2: Application Scenarios

1.8 Dataset Description – COCO 2017

The success of a deep learning segmentation model is intrinsically tied to the quality of its training data. This project utilizes the Common Objects in Context (COCO) 2017 dataset, hosted on Kaggle. COCO is widely regarded as one of the most challenging and comprehensive datasets in computer vision because it features everyday objects captured in highly complex, cluttered, and natural visual contexts, rather than isolated against clean backgrounds.

The dataset provides high resolution images paired with highly detailed, polygon based instance segmentation masks for over 80 object categories. However, utilizing COCO for this specific project required substantial data engineering. Because Vision Extract is designed for binary subject isolation (foreground vs. everything else) rather than multi class identification, the raw, multi class COCO masks could not be used out of the box. A critical phase of the project involved writing preprocessing scripts to parse the COCO annotations and systematically merge or filter them into a single, unified binary mask, setting the ground truth for the model to learn the concept of a generalized "main

subject."

1.9 Existing Technologies and Solutions

Prior to the advent of deep learning, subject isolation relied on a mix of manual tools and basic algorithmic heuristics. Early technologies included global color thresholding (which fails if the subject and background share colors) and edge detection algorithms like the Canny operator (which frequently produces broken boundaries and fails in textured environments). In professional settings, Chroma Keying (green screen technology) was the standard, but it requires highly controlled studio lighting and specific background colors, making it useless for organic, everyday photographs.

Currently, there are several advanced solutions in the market, such as Adobe Photoshop's AI driven "Select Subject" feature or cloud based API services like Remove.bg. While these solutions are highly effective, they operate as proprietary, closed source "black boxes." They offer developers no control over the underlying architecture, limit customization, and often incur significant recurring API costs for bulk processing. The Vision Extract project is designed to bypass these limitations by developing an independent, transparent, and self hosted segmentation pipeline. By building the model from the ground up using the COCO dataset, the project ensures complete control over the data privacy, architectural tuning, and deployment environment, offering a robust alternative to proprietary third party services.

LITERATURE REVIEW

2.1 Introduction to Image Segmentation

Image segmentation is one of the most fundamental and historically significant challenges in the domain of computer vision. At its core, segmentation is the process of partitioning a digital image into multiple contiguous sets of pixels, collectively known as image objects or segments. The primary objective of this partitioning is to simplify the representation of an image, transforming it from a dense matrix of raw pixel intensities into a collection of meaningful, analyzable spatial regions. Unlike broader tasks such as image classification which assigns a singular, global label to an entire image regardless of its internal composition segmentation necessitates a granular, pixel by pixel analysis. Every individual pixel within the input matrix must be evaluated and assigned a discrete categorical label. This detailed understanding of the visual scene allows machines to comprehend not just what objects are present, but exactly where they are located, their precise shapes, and their spatial relationships with surrounding elements. Image segmentation serves as a critical preprocessing step or core computational engine for high level diagnostic, autonomous, and media editing tasks, forming the theoretical bedrock upon which automated subject isolation is built.

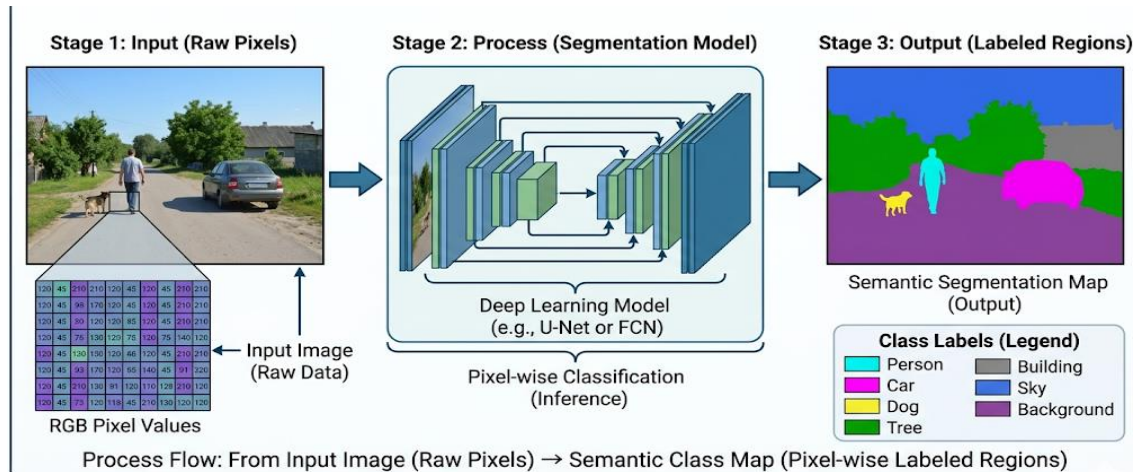


Fig 2.1: Raw Pixels To Labelled Regions

2.2 Traditional Image Segmentation Techniques

Before the widespread adoption and dominance of deep learning architectures, image

segmentation relied heavily on traditional, algorithmic approaches rooted in classical digital image processing. These classical methods generally did not involve "learning" in the modern machine learning sense; rather, they were deterministic algorithms based on mathematical heuristics and the statistical distribution of pixel intensities across an image. While computationally lightweight and highly interpretable, these methods often struggled with generalizability across diverse, complex datasets with heterogeneous lighting, textures, and occlusions.

Thresholding Methods: The simplest and most computationally efficient traditional segmentation technique is thresholding. This method operates on the premise that objects of interest can be separated from the background based solely on their pixel intensity values. In a grayscale image, a threshold value (T) is selected. Pixels with an intensity greater than T are classified as foreground (often assigned a value of 1 or 255), while those with an intensity below T are classified as background (assigned a value of 0). The most famous mathematical formulation of this is Otsu's Method, which calculates an optimal, dynamic threshold by maximizing the inter class variance between the foreground and background pixel distributions. While highly effective for document scanning or highly contrasted medical images, global thresholding fails catastrophically when applied to natural images characterized by uneven illumination or soft, graduated shadows.

Edge Based Segmentation: Recognizing that object boundaries are typically characterized by sudden, sharp changes in pixel intensity, edge based methods attempt to segment images by identifying these discontinuities. Algorithms such as the Canny edge detector and the Sobel operator utilize spatial convolution filters to compute the first or second order derivatives of the image matrix. High derivative values indicate an edge. Once edges are detected, contour linking algorithms attempt to close the boundaries to form coherent segmented regions. However, edge based methods are notoriously sensitive to high frequency image noise and often produce fragmented, discontinuous boundaries that require extensive morphological post processing to yield a closed subject mask.

Region Based Techniques: In contrast to finding boundaries, region based methods focus on growing or splitting homogeneous areas within the image. Region growing starts with a set of user defined or algorithmically selected "seed" pixels. The algorithm iterates over neighboring pixels, appending them to the seed region if they meet a predefined similarity

criterion (such as color variance or texture threshold). Conversely, the region splitting and merging approach starts with the entire image as a single region and recursively divides it into quadrants until each sub region meets the homogeneity criteria, merging adjacent similar regions thereafter. The watershed algorithm, another popular region based method, treats the image gradient as a topographical surface and "floods" it from regional minima to establish catchment basins (segments). While more robust than simple thresholding, region based methods are prone to severe over segmentation in highly textured natural environments.

2.3 Semantic Segmentation

Semantic segmentation represents a monumental leap in the complexity and utility of computer vision tasks. It can be formally defined as the process of assigning a semantically meaningful class label to every single pixel in an image, drawn from a predefined set of categories. Mathematically, for an input image composed of N pixels and a dataset with C discrete classes, a semantic segmentation model must output an $N \times C$ probability distribution matrix. The final categorical mask is typically derived by applying an argmax function across the channel dimension, assigning each pixel to the class with the highest predicted probability.

It is vital to distinguish semantic segmentation from its closely related counterparts: instance segmentation and panoptic segmentation. Semantic segmentation is inherently "stuff aware" but not "thing aware" at the individual level. For example, if an image contains three separate cars parked on a street, a semantic segmentation model will classify all the pixels belonging to the vehicles uniformly as the "car" class, without distinguishing Car A from Car B or Car C. Instance segmentation, conversely, identifies individual object boundaries, differentiating the three cars but typically ignoring amorphous background "stuff" like the road or sky. The Vision Extract project largely operates on the principles of binary semantic segmentation, where the predefined class set (C) is reduced to strictly two categories: the main, unified foreground subject (Class 1) and all surrounding background elements (Class 0).

The challenge of semantic segmentation lies in the inherent tension between global context and local precision. To accurately determine that a patch of pixels belongs to a "dog," the model must have a wide receptive field to understand the global context of the

animal's shape and surrounding environment. However, to accurately draw the boundary of the dog capturing the intricate details of its fur against the grass the model requires high resolution, localized spatial information. Resolving this tension is the primary architectural driver behind modern deep learning segmentation networks.

2.4 Deep Learning in Computer Vision

The integration of deep learning, specifically through artificial neural networks with multiple hidden layers, catalyzed a paradigm shift in computer vision methodology. For decades, the field was bottlenecked by feature engineering. Researchers spent exhaustive efforts manually designing algorithms (like SIFT or HOG) to extract specific visual features such as edges, corners, and color histograms that a downstream classifier (like a Support Vector Machine) could use to make a decision. This handcrafted feature extraction was highly brittle, requiring complete redesigns when transitioning between different visual tasks or datasets.

Deep learning fundamentally inverted this workflow by introducing end to end representation learning. Instead of manually engineering features, data scientists supply a deep neural network with massive quantities of raw data paired with ground truth annotations. Through the iterative process of forward propagation, loss calculation, and backpropagation utilizing gradient descent algorithms, the network autonomously discovers the optimal hierarchical feature representations required to solve the task. The initial layers of a deep network typically act as Gabor like filters, detecting rudimentary lines, edges, and gradients. Deeper layers combine these low level features into intermediate shapes, textures, and object parts. The deepest, final layers synthesize these elements into complex, highly abstract semantic representations of complete objects and scenes. This hierarchical feature abstraction, driven directly by raw data rather than human heuristics, is what allows deep learning models to achieve state of the art performance across nearly all computer vision benchmarks, including classification, detection, and segmentation.

2.5 Convolutional Neural Networks (CNNs)

The Convolutional Neural Network (CNN) is the workhorse architecture of modern computer vision. Traditional feedforward artificial neural networks (Multi Layer Perceptrons) are profoundly ill suited for processing high resolution image data. Flattening

a two dimensional image into a one dimensional vector destroys vital spatial relationships and topological structure, and the requirement for fully connected, dense weights leads to a combinatorial explosion in parameters, resulting in immediate overfitting and computational bottlenecking. CNNs resolve this by leveraging the mathematical operation of convolution, which preserves spatial hierarchy and drastically reduces parameter counts through weight sharing.

Algorithm Class	Key Characteristics	Primary Limitations
Traditional (Otsu, Watershed)	Heuristic based, no learning phase, low computational cost.	Fails on complex textures, varying lighting, lacks semantic understanding.
Standard CNN (Classification)	Learns hierarchical features, robust translation invariance.	Fully connected layers destroy spatial resolution; cannot output pixel masks.
Fully Convolutional Network (FCN)	Replaces dense layers with 1x1 convolutions; outputs spatial maps.	Early upsampling leads to coarse, blurry boundaries without skip connections.

Table 2.1: Characteristics and Limitations of Different Algorithm

A typical CNN architecture is composed of a recurring sequence of specific layer types. **Convolutional Layers** are the core building blocks. They apply a set of small, learnable spatial filters (kernels) that slide across the input image to produce two dimensional activation maps. These maps indicate the spatial presence of specific features. Because the same kernel is applied across the entire image, the model benefits from translation invariance it can recognize a feature regardless of where it appears in the frame. **Activation Layers** introduce non linearity into the network, a crucial requirement for learning complex mappings. The Rectified Linear Unit (ReLU), which zeroes out negative values while passing positive values unchanged, is the standard activation function due to its computational efficiency and its ability to mitigate the vanishing gradient problem during backpropagation. **Pooling Layers** (typically Max Pooling) perform spatial

downsampling. By sliding a window across the activation map and taking only the maximum value within that window, pooling reduces the spatial dimensions of the data. This controls overfitting, reduces computational load, and increases the receptive field of subsequent convolutional layers, allowing them to capture a wider global context. In traditional CNNs designed for image classification, the architecture concludes with **Fully Connected (Dense) Layers**, which flatten the spatial feature maps into a vector to output a final probability distribution over the target classes. However, for pixel wise tasks like segmentation, these fully connected layers present a critical flaw: they destroy all spatial resolution.

2.6 Encoder–Decoder Architectures

To adapt the powerful feature extraction capabilities of CNNs for the task of semantic segmentation, researchers developed the Encoder Decoder architecture, fundamentally popularized by the Fully Convolutional Network (FCN) paradigm. The primary innovation of the FCN was the complete removal of dense, fully connected layers, replacing them with 1x1 convolutions. This allowed the network to accept input images of arbitrary sizes and output two dimensional spatial classification maps rather than a single vector.

The Encoder Decoder structure elegantly addresses the tension between context and localization. The **Encoder** half of the network acts as a standard classification CNN (such as VGG16 or ResNet) stripped of its final dense layers. Its role is feature extraction. Through repeated blocks of convolution and max pooling, the encoder progressively reduces the spatial resolution of the input image (width and height) while simultaneously expanding the channel dimension (depth) to capture complex, highly abstract semantic features. By the time the data reaches the "bottleneck" at the end of the encoder, it is a highly compressed, low resolution tensor possessing deep "what" information (context), but having lost critical "where" information (localization). The **Decoder** half of the network is responsible for spatial reconstruction. It receives the heavily downsampled feature map from the encoder and systematically projects it back up to the original image resolution. This is achieved through upsampling techniques, such as bilinear interpolation or learnable transposed convolutions (often referred to as deconvolution). As the spatial dimensions increase, the number of feature channels is progressively reduced, culminating in a final output layer that matches the height and width of the original image, with a

channel depth corresponding to the number of target segmentation classes.

2.7 U Net Architecture

While early FCNs successfully generated segmentation masks, their outputs were notoriously coarse and lacked crisp boundary definitions. Because the decoder was attempting to rebuild high resolution boundaries purely from the heavily compressed bottleneck features, intricate spatial details were irrevocably lost. The U Net architecture, introduced by Ronneberger et al. in 2015, provided a revolutionary solution to this problem and remains the foundational benchmark for medical and general image segmentation tasks.

U Net is a symmetric, U shaped encoder decoder network distinguished by its innovative use of **Skip Connections**. The architecture consists of a contracting path (encoder) and an expansive path (decoder). The contracting path follows the typical architecture of a convolutional network, applying repeated 3x3 unpadded convolutions, each followed by a ReLU activation and a 2x2 max pooling operation for downsampling, doubling the number of feature channels at each step. The expansive path mirrors this structure, but instead of pooling, it utilizes 2x2 transposed convolutions for upsampling, halving the number of feature channels.

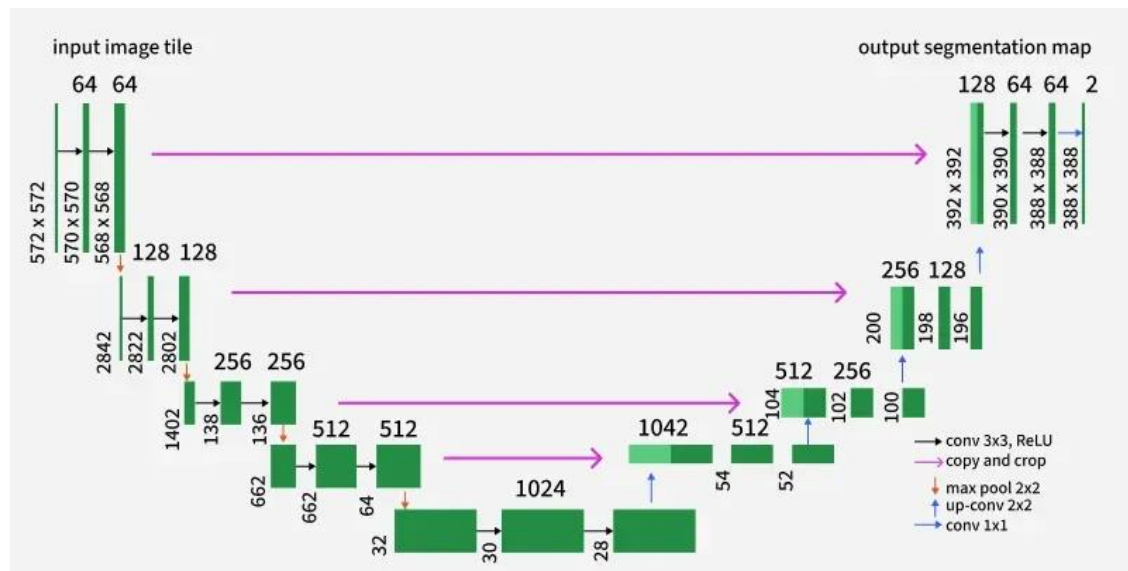


Fig 2.2: symmetric U Net Architecture with Skip Connections

The brilliant modification of the U Net lies in the skip connections. At every corresponding spatial level between the contracting and expansive paths, the high

resolution, localized feature maps from the encoder are cropped (if necessary) and directly concatenated across the channel dimension with the upsampled feature maps in the decoder. By explicitly routing this raw, high frequency spatial data past the bottleneck and feeding it directly to the reconstruction layers, the decoder is provided with the exact coordinate information required to draw precise boundaries. The subsequent convolutional layers in the decoder then learn how to assemble a precise output mask by fusing the deep, abstract semantic context (coming up from the bottleneck) with the fine grained, localized spatial details (coming across via the skip connections). This elegant fusion makes U Net exceptionally powerful even when trained on highly constrained, small datasets.

2.8 LinkNet and DeepLabV3+

LinkNet Architecture: While U Net provides excellent segmentation accuracy, the operation of concatenating feature maps across skip connections drastically inflates the channel depth in the decoder, requiring substantial computational resources and memory overhead. LinkNet was proposed as a highly efficient alternative designed specifically for real time semantic segmentation on resource constrained devices, without significantly sacrificing accuracy.

LinkNet maintains the standard encoder decoder topology but introduces a critical modification to the bypass mechanism. Instead of concatenating the feature maps from the encoder to the decoder which doubles the channel width and thereby quadruples the parameters in the subsequent convolutional layers LinkNet utilizes an addition (element wise summation) operation. The encoder feature map is simply added to the corresponding upsampled decoder feature map. By bypassing the concatenation step, LinkNet preserves the essential high resolution spatial data for boundary recovery while maintaining a significantly leaner parameter profile, resulting in vastly faster inference times suitable for applications like real time video processing and autonomous navigation.

DeepLabV3+ Architecture: DeepLabV3+ represents one of the most advanced state of the art architectures in semantic segmentation, addressing the limitations of standard pooling and convolution operations through sophisticated spatial manipulations. The core innovation of the DeepLab series is the extensive use of Atrous (or dilated) convolutions. In a standard convolution, the kernel filters adjacent pixels. In an atrous convolution, the kernel filter is expanded by inserting "holes" (zeros) between the filter weights, effectively

dilating the kernel's field of view without increasing the number of learnable parameters or aggressively downsampling the spatial resolution of the feature map.

DeepLabV3+ couples these dilated convolutions with an Atrous Spatial Pyramid Pooling (ASPP) module. The ASPP applies multiple parallel atrous convolutions with varying dilation rates to the feature map extracted by the encoder backbone. This allows the network to probe the incoming visual data at multiple scales simultaneously, capturing multi scale contextual information crucial for accurately segmenting objects that appear at drastically different sizes within the same image. The "plus" in DeepLabV3+ denotes the addition of a simple yet highly effective decoder module (similar to U Net's approach) that refines the segmentation results, particularly along object boundaries, which were somewhat degraded in earlier DeepLab iterations that lacked a dedicated spatial recovery mechanism.

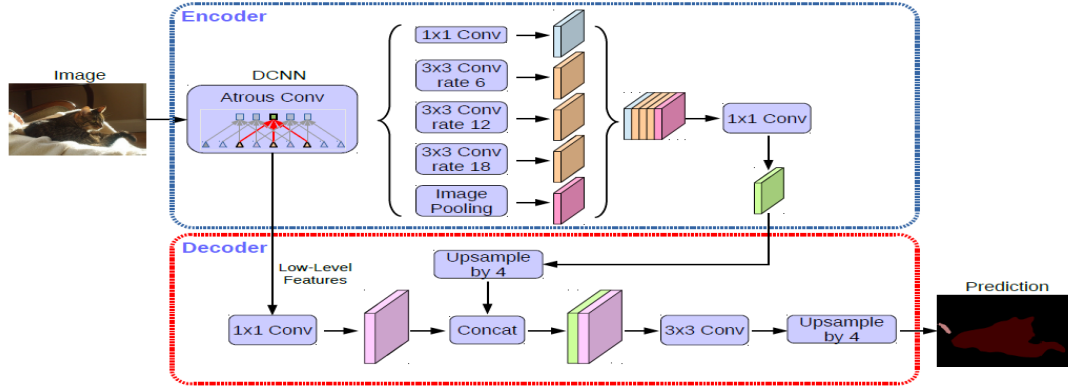


Fig 2.3: DeepLabV3+ Architecture

2.9 Uniqueness of the Proposed System

While several robust commercial systems exist for subject isolation, the proposed Vision Extract project distinguishes itself through its specific focus, architectural transparency, and end to end integration strategy designed for an educational and specialized deployment context.

Firstly, many existing background removal tools are "black box" proprietary systems or cloud based API services that charge per request and offer little insight into their underlying mechanics or data handling policies. The proposed system is entirely transparent, documenting the full pipeline from raw data ingestion to model deployment. This white box approach is crucial for understanding the limitations and biases inherent in

the training data, allowing for targeted improvements rather than relying on an opaque third party service.

Secondly, unlike general purpose models attempting panoptic segmentation (identifying every object in the scene), this system is hyper optimized for the singular task of binary subject extraction using a specialized preprocessing pipeline on the complex COCO 2017 dataset. By actively converting multi class masks into strict binary foreground/background representations during the data engineering phase, the model is forced to learn saliency the concept of what constitutes a "main subject" in a cluttered scene rather than simply memorizing specific object classes. This improves the model's zero shot capability when confronted with novel, unclassified objects that are clearly acting as the primary focal point of an image.

Finally, the uniqueness lies in the complete, full stack implementation built as a standalone service. By integrating the optimized image segmentation model directly into a custom Backend REST API connected to a dedicated web application interface, the project provides a self hosted, scalable solution that processes image blobs and returns the isolated results with a rendered black background. This demonstrates not just a theoretical machine learning experiment, but a tangible software engineering product ready for integration into broader media workflows or localized deployment environments.

DEVELOPMENT METHODOLOGY

3.1 System Architecture Overview

The development methodology for the Vision Extract project follows a structured, modular approach designed to transition seamlessly from large scale data engineering to real time model deployment. This methodology is grounded in the principles of software engineering for machine learning (MLOps), ensuring that each phase from raw data ingestion to the final REST API response is reproducible, scalable, and optimized for high performance computing environments. The overarching system architecture is composed of three primary, interconnected phases: the Data Pipeline, the Model Architecture, and the Inference Architecture. By decoupling these phases, the system maintains a high degree of flexibility, allowing for iterative improvements to the neural network without necessitating a redesign of the underlying data handling or deployment logic.

The **Data Pipeline** is the foundational engine responsible for ingesting the raw COCO 2017 dataset from local or cloud storage. It executes the critical binary mask conversion algorithm, transforming complex polygon metadata into spatially aligned raster masks. Beyond simple conversion, this pipeline manages the orchestration of data loaders, executing multi threaded preprocessing and stochastic augmentation steps to format the data into high dimensional consumable tensors for the neural network. The **Model Architecture** constitutes the core computational engine, specifically a custom designed U Net Convolutional Neural Network (CNN) built using the PyTorch framework. This architecture utilizes a hierarchy of convolutional filters to extract features at multiple scales, from low level edges to high level semantic objects. Finally, the **Inference Architecture** represents the production deployment phase. In this stage, the optimized model weights are encapsulated within a containerized backend REST API developed with Flask or FastAPI. This service is designed to receive incoming image blobs from a client application, perform the segmentation inference in a low latency environment, and return the isolated subject on a mathematically pure black background (RGB [0,0,0]).

This modular design ensures that each component can be developed, tested, and optimized independently. For instance, the data augmentation strategies can be heavily modified

shifting from simple flips to complex elastic transforms or GAN generated noise without requiring any structural changes to the underlying REST API. Similarly, the U Net architecture can be swapped for a DeepLabV3+ model or a SegFormers transformer based model without altering the data ingestion pipeline, provided the tensor input shapes remain consistent. This abstraction provides a future proof framework where the project can evolve alongside advancements in computer vision research while maintaining a stable production baseline.

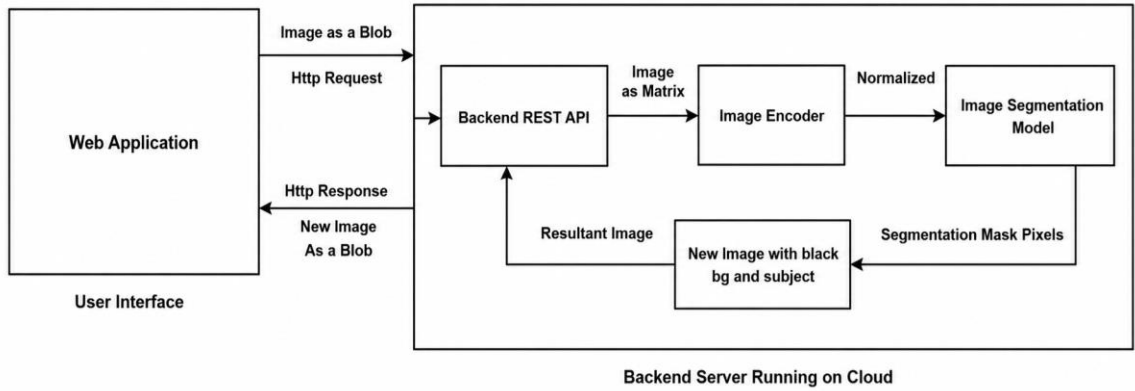


Fig 3.1: System Architecture

3.2 Data Collection and Dataset Preparation

The foundation of the Vision Extract model is the Common Objects in Context (COCO) 2017 dataset, which serves as a global benchmark for object detection and instance segmentation. Accessed via the Kaggle platform, this dataset includes over 200,000 labeled images. The dataset was selected due to its unparalleled complexity compared to more sanitized datasets like MNIST or CIFAR 10. Unlike controlled datasets with uniform backgrounds or centered subjects, COCO features highly cluttered, real world scenes where objects appear in natural contexts. It includes challenging scenarios such as heavy occlusion (one object blocking another), diverse subject scales (from tiny birds to large buildings), and varying environmental conditions (rain, shadows, and low light). This complexity is crucial for training a robust subject isolation model capable of generalizing to unpredictable user uploads in a consumer facing application.

The initial data collection phase involved downloading the high resolution training, validation, and test image sets, totaling over 25 gigabytes of compressed data. The corresponding annotation files utilize a massive JSON schema containing specific polygon

coordinates (x,y pairs) that map the exact pixel boundaries of 80 different object categories. The preparation phase required writing custom Python scripts utilizing the *pycocotools* library to efficiently parse these JSON structures. The technical objective was to create a structured local file system where every RGB input image was strictly paired with a corresponding high fidelity mask image file. This ensured that the critical image to mask spatial alignment required for supervised learning was maintained. Furthermore, a validation script was implemented to purge any corrupted images or empty annotations, ensuring that the training cycle would not be interrupted by data level exceptions or gradient explosions caused by invalid inputs.

3.3 Binary Mask Generation

The most critical step in the dataset preparation was the translation of raw COCO annotations into a format suitable for the specific goals of the Vision Extract project. The COCO dataset is inherently designed for multi class instance segmentation meaning if a picture contains a dog, a person, and a car, the JSON annotation provides separate polygon boundaries and class IDs for each distinct object. For a general segmentation model, this is useful; however, for a tool focused on "extracting" a single subject, these multi class annotations create ambiguity. The model must be taught to differentiate between the primary subject of interest and secondary objects that should be treated as part of the background.

Because this project focuses on binary subject isolation (Foreground vs. Background), these multi class annotations had to be converted into unified binary masks through a custom built heuristic algorithm. This algorithm iterates through every image's annotation metadata to determine the "main subject." The primary subject was identified based on two priority factors: semantic relevance (specifically filtering for the 'person' class for portrait mode isolation) and geometric dominance (selecting the annotation with the largest bounding box area). Once the primary subject was identified, its polygon coordinates were rendered onto a blank 2D canvas as white pixels (value 255 or 1), while all other objects, even those labeled in COCO, were treated as negative space and rendered as black pixels (value 0). This step was absolutely vital; it forcibly restructured the dataset to teach the neural network to look for a singular salient subject, effectively performing "salient object detection" rather than exhaustive multi label classification. This reduces the

complexity of the final softmax layer and focuses the model's capacity on refining the edges of the primary subject.

3.4 Data Preprocessing Pipeline

Neural networks require uniform, standardized numerical inputs to function effectively and avoid mathematical instability during the training process. The raw images in the COCO dataset vary drastically in their spatial resolutions ranging from low resolution thumbnails of 320x240 to high resolution captures of 1024x768 and feature varying aspect ratios (portrait, landscape, and square). Training a model on varied shapes would require complex dynamic padding or spatial pyramid pooling, which adds computational overhead. To mitigate this, a rigorous data preprocessing pipeline was implemented to standardize all inputs into a consistent format suitable for the GPU's memory architecture.

The first step involved **Resolution Consistency**. Every input RGB image and its corresponding generated binary mask were resized to a fixed spatial dimension of 256x256 pixels. While higher resolutions (e.g., 512x512) provide more detail, 256x256 was chosen as the optimal balance between edge precision and training speed. It was critical that the interpolation method used for resizing the mask differed from the image. The RGB image was resized using bilinear or bicubic interpolation to preserve smooth color gradients and high frequency details. In contrast, the binary mask was resized using nearest neighbor interpolation. This ensures the mask values remained strictly binary (0 or 1), preventing the creation of anti aliased "gray" pixels along the boundaries which would confuse the loss function during pixel wise classification.

The second step was **Input Normalization and Standardization**. The RGB pixel values, originally ranging from 0 to 255, were first scaled down to a floating point range between 0.0 and 1.0. Following this, standard ImageNet normalization was applied, subtracting the mean ([0.485, 0.456, 0.406]) and dividing by the standard deviation ([0.229, 0.224, 0.225]) for each color channel. This process centers the data distribution around zero with unit variance. Such normalization significantly accelerates the convergence of the gradient descent optimization algorithm (Adam) and mitigates the risk of vanishing or exploding gradients. Furthermore, it ensures that the model is more responsive to structural features rather than global changes in image brightness, leading to more consistent performance across different camera sensors and lighting environments.

3.5 Data Augmentation Techniques

To enhance the robustness of the model and prevent overfitting a common pitfall where the network memorizes the specific pixel patterns of the training set rather than learning generalizable semantic features extensive data augmentation was integrated into the pipeline. Augmentation artificially expands the diversity of the dataset by applying random, realistic transformations to the images in real time during each training epoch. This ensures that the model rarely sees the exact same image twice, forcing it to develop invariant features that are resistant to changes in orientation, scale, and lighting. This is particularly important for the Vision Extract project, where user submitted photos may be taken at awkward angles or in suboptimal lighting.

The augmentation suite was categorized into spatial and pixel level transforms. Crucially, spatial augmentations such as random horizontal flipping ($p=0.5$), random rotations (± 20 degrees), and random scale crops were applied simultaneously and identically to both the input RGB image and its corresponding mask. This synchronization is paramount; if the image is rotated while the mask remains static, the labels become invalid. This ensures the spatial relationship and boundary alignment between the image and the mask are perfectly preserved. Furthermore, pixel level augmentations, such as random color jitter (varying brightness, contrast, and saturation by 20%) and the addition of Gaussian noise, were applied exclusively to the RGB image. These color perturbations force the model to learn the structural and semantic boundaries of the subject rather than relying on specific lighting conditions or color palettes, leading to a model that can isolate a subject regardless of whether the photo was taken at noon or at sunset.

3.6 Custom PyTorch Dataset Class

To efficiently manage the high throughput data loading process required for deep learning, a custom Dataset class was engineered within the PyTorch framework. This class inherits from the *torch.utils.data.Dataset* base class and overrides the essential `__len__` and `__getitem__` methods. The design of this class focuses on lazy loading images are only loaded into memory when they are needed for the current training batch, rather than loading the entire 25GB dataset into RAM. This architectural choice allows the training process to scale to datasets much larger than the available system memory.

When the training loop requests a batch of data, the `__getitem__` method fetches the

specific image and mask file paths from the disk based on a shuffled index. It utilizes the PIL (Python Imaging Library) to read the files, applies the defined sequence of stochastic augmentations via the Albumentations library, and executes the resizing and normalization steps. Finally, it converts the resulting numpy arrays into PyTorch Tensors. The RGB image is formatted as a FloatTensor with dimensions [3, 256, 256] (following the NCHW convention), while the mask is formatted as a LongTensor representing binary class indices. This custom class was then wrapped in a PyTorch *DataLoader*. The DataLoader was configured with ``num_workers=4`` to enable asynchronous multi processing, which pre fetches and prepares the next batch on the CPU while the current batch is being processed on the GPU. This eliminates CPU bottlenecks and ensures maximum utilization of the hardware, significantly reducing the total wall clock time for model training.

3.7 Custom U Net CNN Architecture

The core segmentation model was implemented as a custom U Net architecture, a specialized type of Fully Convolutional Network (FCN). Originally developed for biomedical image segmentation, the U Net was chosen for the Vision Extract project due to its proven efficacy in generating high resolution, pixel perfect masks even in complex environments. The architecture is defined by its symmetric "U" shape, consisting of a contracting encoder path that captures context and an expansive decoder path that enables precise localization. This structure is uniquely suited for subject isolation because it can understand "what" an object is while simultaneously knowing exactly "where" its boundaries are located at the pixel level.

The Image Encoder (Contracting Path): The encoder consists of four primary blocks. Each block applies two consecutive 3x3 convolutional layers with padding to maintain spatial dimensions. Each convolution is followed by Batch Normalization (BN) and a Rectified Linear Unit (ReLU) activation function. BN is critical for stabilizing the internal covariate shift, allowing for higher learning rates and faster convergence. After the dual convolutions, a 2x2 Max Pooling operation is applied, which downsamples the spatial resolution by half (e.g., $256 > 128 > 64 \dots$) while doubling the number of feature channels (e.g., $64 > 128 > 256 \dots$). This path acts as a hierarchical feature extractor; as the spatial resolution decreases, the network's receptive field increases, allowing it to capture deep semantic context and global object shapes.

The Decoder (Expansive Path): The decoder mirrors the encoder, utilizing four expansive blocks to recover the original spatial resolution. Each block begins with a 2x2 Transposed Convolution (also known as a deconvolution), which upsamples the feature maps by a factor of two while halving the channel depth. Crucially, before the subsequent convolutions, the corresponding high resolution feature map from the encoder is concatenated along the channel dimension with the upsampled map. These skip connections are the defining innovation of the U Net; they provide a direct path for low level spatial coordinate data (edges and textures) to reach the final layers, bypassing the semantic bottleneck. This ensures that the final mask is not just a blurry blob but a sharp, edge aligned isolation of the subject. The architecture concludes with a final 1x1 convolution mapping the deep 64 channel feature set to the 2 target output classes. A sigmoid activation function is applied at inference time to generate a probability map, where values close to 1.0 represent the foreground subject.

3.8 Model Selection and Training

While the custom U Net was the primary architecture, extensive experiments were conducted using Transfer Learning. In these experiments, the custom encoder was replaced with a ResNet 34 backbone pre trained on the ImageNet 1K dataset. Leveraging a pre trained backbone allows the model to utilize highly developed, generalized feature extractors (capable of recognizing edges, gradients, and shapes) immediately upon initialization. This approach significantly reduces the number of required training epochs, as the model only needs to learn the task specific "decoder" logic rather than learning how to see from scratch. Comparative testing showed that the ResNet 34 backbone reached a 0.85 IoU faster than the custom encoder, particularly in complex images containing varied textures like fur or foliage.

The training process was executed using a standard supervised learning loop on an NVIDIA T4 GPU. For each batch (batch size of 32), the input tensors were passed forward through the network to generate raw prediction logits. These logits were compared against the ground truth binary masks to calculate the error (loss). The loss was then backpropagated through the network using AutoGrad to calculate gradients, and the Adam optimizer updated the network's weights with a base learning rate of $1e-4$. A learning rate scheduler, specifically 'ReduceLROnPlateau', was implemented to monitor validation

loss; if the loss stalled for 5 epochs, the learning rate was reduced by a factor of 10. This allowed the model to make coarse adjustments early in training and finer, surgical weight updates as it approached a global minimum. Training was monitored using TensorBoard, tracking loss curves and visually inspecting prediction masks at the end of each epoch. Early stopping was also employed, halting the process if the validation IoU did not improve for 15 consecutive epochs, preventing the model from over fitting to the training distribution.

3.9 Loss Functions and Optimizers

The selection of an appropriate loss function is paramount in semantic segmentation, particularly for tasks prone to class imbalance. In subject isolation, the "background" pixels often vastly outnumber the "foreground" pixels (e.g., a small person standing in a large field). If the model were trained using only a standard Cross Entropy loss, it could achieve a misleadingly high accuracy of 90% simply by predicting every pixel as the background. This would result in a failure of the core project goal effective subject isolation. To combat this, the loss function must penalize false negatives (missed subject pixels) as heavily as false positives, regardless of the area they occupy.

To ensure balanced learning, a composite loss function was utilized, combining **Binary Cross Entropy (BCE)** with **Dice Loss**. BCE measures the pixel wise classification error independently, which is excellent for ensuring high probability confidence in stable regions. Dice Loss, derived from the Sorensen Dice Coefficient, evaluates the degree of spatial overlap between the predicted mask (P) and the ground truth (G). Because Dice Loss looks at the intersection divided by the sum of the areas, it is inherently robust to class imbalances; it cares about the shape of the subject, not just the pixel count. The final loss with weights tuned to 0.5 each. The Adam optimizer (Adaptive Moment Estimation) was selected to navigate this loss landscape, as its ability to maintain per parameter learning rates allows it to handle the different gradients produced by the BCE and Dice components effectively.

3.10 Evaluation Metrics (IoU and Dice Score)

Standard classification accuracy, which measures the percentage of correctly classified pixels, is an inadequate and often deceptive metric for segmentation. In a typical image where the background occupies 95% of the frame, a model that predicts "background" for

everything would have 95% accuracy but 0% utility. Therefore, the Vision Extract model's performance was strictly evaluated using spatial overlap metrics that specifically focus on the quality of the foreground isolation: the Intersection over Union (IoU) and the Dice Score. These metrics provide a rigorous assessment of how well the model's predicted boundaries align with the actual subject boundaries.



Fig 3.2: Evaluation and Result

Intersection over Union (IoU), also known as the Jaccard Index, is the gold standard for evaluating segmentation models. It is calculated by dividing the area of overlap between the predicted mask and the ground truth mask by the total area covered by both (the union). An IoU score of 1.0 represents a perfect pixel for pixel match, while 0.0 indicates absolutely no overlap. IoU is exceptionally strict; any "stray" pixels or missing edges significantly penalize the score. This makes it an ideal metric for the Vision Extract project, as even small errors in a subject's boundary (like missing part of an arm) are highly visible to a user and must be minimized. Our target threshold for deployment was a Mean IoU (mIoU) of 0.82 across the validation set.

The Dice Score (or Dice Coefficient) is closely related to IoU but is mathematically weighted differently. It is calculated as twice the area of overlap divided by the sum of the individual areas. While IoU is a more punishing metric, the Dice Score provides a smoother representation of the model's performance, acting as the harmonic mean of precision and

recall. Because the Dice Score is also the foundation of our loss function (Dice Loss), monitoring it on the validation set provides a direct check on the model's optimization progress. By calculating both IoU and Dice Score, we gain a comprehensive view of the model's performance. IoU tells us how perfect the boundaries are, while the Dice Score tells us how well the model is capturing the overall mass of the subject. These metrics were computed at the end of each epoch, and only the weights that achieved the highest validation IoU were saved for the final production pipeline.

TECHNOLOGY STACK

4.1 Programming Language – Python

The entirety of the Vision Extract project, spanning from data preprocessing scripts to the backend deployment API, was engineered using Python. Python is currently the de facto lingua franca for artificial intelligence, machine learning, and data science. Its selection was driven by three primary factors: its syntactical readability, its unparalleled ecosystem of specialized scientific libraries, and its robust community support.

Unlike languages such as C++ or Java, which require extensive boilerplate code, Python's highly readable syntax allows researchers and engineers to focus on the mathematical logic of the algorithms rather than the intricacies of memory management. This characteristic is particularly crucial in deep learning, where rapid prototyping and iterative experimentation are the norm.

The implementation relied heavily on the integration of core scientific libraries beyond the deep learning framework itself. Specifically, the NumPy library formed the backbone for handling array and matrix operations during the initial data pipeline stage, prior to converting images into PyTorch tensors. This included essential tasks like efficient vectorization of operations, which is crucial for maximizing computational speed, especially for operations like mean subtraction and standard deviation scaling.

The choice of Python streamlined the project across its three main phases. In the Data Engineering phase, Python facilitated the construction of custom data loaders and classes (`'Dataset'` and `'DataLoader'` in PyTorch) which abstracted the complexity of data retrieval and transformation. During the Model Training phase, Python's dynamic nature, combined with PyTorch's eager execution, enabled highly detailed runtime debugging and instantaneous model inspection, a necessity when iterating on custom U Net architectures. Finally, in the Deployment phase (detailed in Section 4.7), the robust library support made Python the ideal choice for creating a lightweight REST API using Flask, handling HTTP requests, image deserialization, model inference calls, and final image synthesis within a unified, easily maintainable codebase. This uniformity across the stack prevented the overhead associated with managing dependencies or type conversions across multiple

programming languages.

4.2 CPU vs GPU Utilization

A critical architectural consideration in deep learning is the allocation of computational resources, specifically the dichotomy between the Central Processing Unit (CPU) and the Graphics Processing Unit (GPU). Understanding the hardware utilization was essential for optimizing the training pipeline of the Vision Extract model.

The Role of the CPU is designed for sequential processing and handles complex logic and branching operations exceptionally well. In the context of this project, the CPU was strictly tasked with the Data Engineering pipeline. This included reading raw image files from the disk (I/O operations), executing JSON parsing for the COCO dataset annotations, performing data augmentations (like cropping and color jittering), and batching these modified images into tensors. This creates a multi threaded data loading pipeline (often utilizing PyTorch's `'num_workers'` parameter) that continuously prepares data asynchronously.

The effective utilization of the CPU in the data pipeline is paramount to prevent the GPU from becoming *data starved*, a common performance bottleneck in deep learning. The multi threaded pipeline, managed by the PyTorch `'DataLoader'` and the `'num_workers'` parameter, was carefully tuned to leverage available CPU cores for parallel data preparation. This involves maximizing the efficiency of disk I/O and preprocessing routines, ensuring that a new batch of pre processed tensors is ready in RAM the moment the GPU finishes processing the previous batch. Improper tuning such as setting `'num_workers'` too low forces the GPU to idle, wasting computational resources. Conversely, setting it too high can lead to excessive context switching overhead on the CPU, diminishing returns. The optimal configuration was determined through profiling to achieve a balance where the CPU throughput matched the GPU's consumption rate.

The Role of the GPU is designed for parallel processing, consisting of thousands of smaller, specialized cores capable of executing identical mathematical operations simultaneously across massive datasets. Training a U Net architecture requires computing millions of matrix multiplications (convolutions) during forward propagation and calculating complex gradients during backpropagation. Executing this on a CPU would take weeks per epoch. Therefore, the GPU was exclusively utilized for the Model

Architecture phase. Once the CPU prepared a batch of tensors, they were transferred to the GPU's VRAM using the `.to(device)` command, where the neural network weights resided. This division of labor ensures that the GPU is never idle, waiting for data, maximizing the overall throughput of the training process.

GPU utilization is fundamentally constrained by the size of the VRAM (Video Random Access Memory). For the Vision Extract project, which involves high resolution image inputs and large U Net models, careful VRAM management was essential. The batch size a critical hyperparameter was primarily determined by the maximum size that could fit within the available VRAM, as increasing the batch size directly improves the parallelism efficiency of the GPU. Furthermore, the use of half precision floating point formats (FP16) was explored using PyTorch's Automatic Mixed Precision (AMP) utilities. AMP dynamically scales loss values to maintain numerical stability while halving the memory footprint of both the model weights and the activations, significantly boosting the memory capacity and, consequently, the maximum effective batch size. The transfer of data from CPU RAM to GPU VRAM via the PCIe bus is itself a bottleneck; hence, minimizing unnecessary transfers and performing all complex, large scale matrix operations directly on the GPU device were prioritized. The `device = torch.device('cuda')` assignment was a core component of this strategy (see Section 4.8).

4.3 Deep Learning Framework – PyTorch

PyTorch, developed primarily by Meta AI (formerly Facebook AI Research), was selected as the core deep learning framework for this project over alternatives like TensorFlow/Keras. The decision was based on PyTorch's imperative programming paradigm, specifically its dynamic computational graph generation, which is highly advantageous for research and custom architecture development.

Unlike older frameworks that require compiling a static graph before any data is passed through, PyTorch builds the computational graph on the fly as the code is executed line by line. This "eager execution" makes debugging significantly easier, as developers can use standard Python print statements to inspect tensor shapes and values at any point within the neural network's forward pass. Furthermore, PyTorch provides native, highly optimized modules for convolutions (`torch.nn.Conv2d`), transposed convolutions for the expansive decoder path (`torch.nn.ConvTranspose2d`), and sophisticated loss functions

required for the segmentation task. The framework's `autograd` engine automatically computes the complex partial derivatives required for backpropagation, abstracting away the intense calculus involved in updating the U Net weights.

The concept of a dynamic graph proved invaluable during the iterative design of the U Net's skip connections and expansive path. Since the U Net architecture requires concatenating feature maps from the encoder path to the decoder path, tracking the precise tensor shape ($B \times C \times H \times W$, i.e., Batch, Channels, Height, Width) at every concatenation point is essential. With PyTorch's eager mode, shape mismatches or logic errors could be detected immediately using standard Python debugging tools like breakpoints or print statements, rather than waiting for a complex, pre compiled static graph to fail after hours of processing. This iterative freedom significantly shortened the development cycle for architectural experimentation.

The PyTorch `autograd` system is the engine that facilitates gradient calculation. Every `torch.Tensor` object carries a `.grad\_fn` attribute, which references the function that created it (e.g., a convolution or ReLU operation). When a tensor is defined, PyTorch starts tracking the sequence of operations applied to it, creating a chain of dependency nodes (the computational graph). To initiate backpropagation, the loss tensor's `.backward()` method is called. This traverses the graph backwards from the loss function to the input data, calculating the partial derivatives (gradients) at each node using the chain rule of calculus. These gradients are then accumulated in the `.grad` attribute of the leaf tensors (the trainable weights). For the U Net segmentation task, where the loss function (often a combination of BCE and Dice Loss) is highly complex, this automatic differentiation capability is non negotiable, as manually deriving the gradients for millions of parameters would be impossible. The `torch.optim` modules, such as Adam or SGD, then utilize these computed gradients to update the model weights.

4.4 OpenCV for Real Time Segmentation

OpenCV (Open Source Computer Vision Library) is an industry standard library utilized extensively throughout the data pipeline and inference stages of this project. While PyTorch handles the neural network computations, OpenCV is the workhorse for all foundational image manipulation tasks.

During the dataset preparation phase, OpenCV was used to read images from the disk into

NumPy arrays. Crucially, OpenCV reads images in the BGR (Blue, Green, Red) color format by default, necessitating immediate conversion to the standard RGB format expected by most deep learning models (`cv2.cvtColor`). Furthermore, OpenCV's highly optimized resizing functions were critical; as discussed in the methodology, `cv2.INTER_LINEAR` was utilized for resizing RGB images to maintain smooth gradients, while `cv2.INTER_NEAREST` was strictly enforced when resizing the binary mask annotations to prevent interpolation artifacts along the subject boundaries.

The real time segmentation deployment necessitates a robust post processing workflow executed via OpenCV. After the model produces a probability map (a floating point tensor where values range from 0.0 to 1.0), this is converted to a discrete binary mask. This is typically done through thresholding; a fixed threshold (e.g., 0.5) is often employed, but dynamic thresholding methods like Otsu's binarization can sometimes yield better results by automatically determining the optimal separation point. To refine the edges of the predicted mask, morphological operations were employed. For instance, a small kernel based erosion operation was sometimes used to remove small, isolated noise speckles (false positives), while dilation could be used to fill small holes in the predicted segmentation (false negatives).

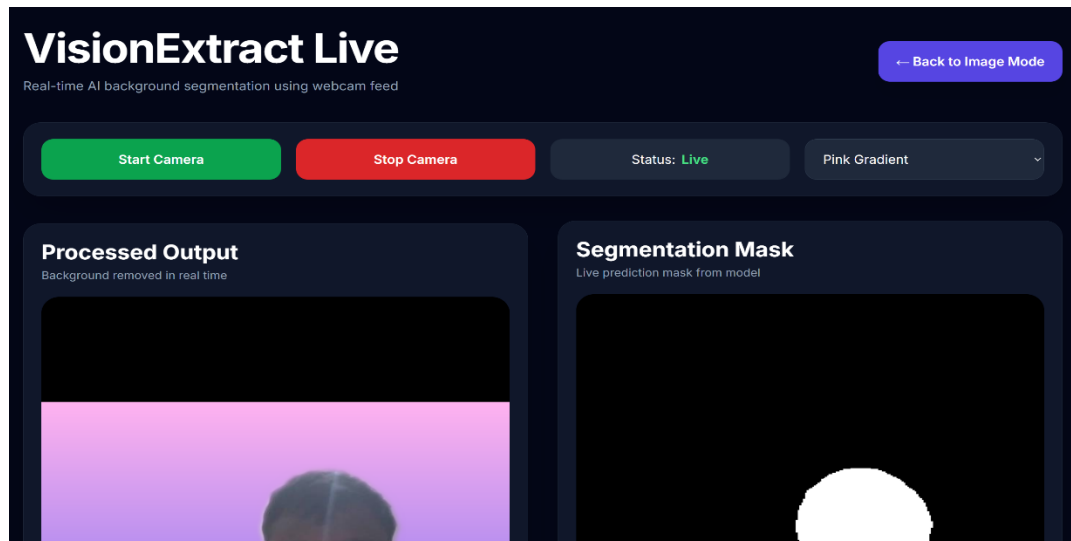


Fig 4.1: Real Time Segmentation web App

The final step is the synthesis of the output image, which must be accomplished without introducing visual artifacts. The `cv2.bitwise_and` operation is performed between the original high resolution input image and a mask derived from the predicted binary

mask. Specifically, the original image is first separated into its B, G, and R color channels. The final binary mask is then applied to each channel independently using the bitwise operation, which results in keeping only the pixels that correspond to the foreground (where the mask value is 1) and zeroing out all background pixels. The three resulting masked channels are then merged back together to form the final RGB image with a transparent or black background, ready for transmission via the Flask API response. This precise control over pixel level manipulation is what makes OpenCV indispensable in the Interface.

4.5 Albumentations and Data Augmentation

To artificially expand the training dataset and improve model generalization, the Albumentations library was integrated into the PyTorch 'Dataset' class. While PyTorch provides native augmentation tools via 'torchvision', Albumentations is significantly faster and, crucially, is specifically designed for complex computer vision tasks like semantic segmentation and object detection.

The primary advantage of Albumentations is its ability to apply spatial transformations consistently across multiple targets simultaneously. When an image is randomly rotated or horizontally flipped to create a new training example, the corresponding binary mask must undergo the exact same geometric transformation to maintain alignment. Albumentations handles this complex synchronization seamlessly through a single API call, preventing the catastrophic misalignment that often occurs when attempting to write custom augmentation loops. The project utilized this library to construct a robust augmentation pipeline including 'HorizontalFlip', 'ShiftScaleRotate', 'RandomBrightnessContrast', and 'GridDistortion', forcing the U Net model to learn the invariant spatial features of a subject rather than memorizing the exact pixel layout of the training data.

In computer vision tasks like classification, augmentation (such as flipping or cropping) only needs to be applied to the input image. For pixel level tasks such as semantic segmentation, however, the target annotation (the binary mask) is intrinsically spatial. Any geometric transformation applied to the input image must be precisely replicated on the mask. A misalignment by even a single pixel would result in the model learning erroneous gradient information during backpropagation, teaching it that a pixel belonging to the

foreground is actually background, leading to training instability and poor boundary prediction. Albumentations solves this by internally handling the affine transformation matrix calculation and applying it to both the image tensor and the mask tensor simultaneously, a feature that significantly differentiates it from simpler augmentation libraries.

Detailed Analysis of Augmentations Used

- **HorizontalFlip (p=0.5):** This is a simple but effective technique that doubles the dataset size in terms of left/right orientation variation. It compels the model to recognize subjects regardless of their horizontal facing direction.
- **ShiftScaleRotate (shift_limit=0.0625, scale_limit=0.1, rotate_limit=15, p=0.5):** This function applies small random affine transformations. The shift introduces minor translations, preventing the model from fixing the subject's position to the center of the frame. The scale variance forces the model to perform robust segmentation across slightly different subject sizes. The rotation of up to 15 degrees encourages rotational invariance.
- **RandomBrightnessContrast (p=0.2):** This augmentation simulates varying lighting conditions, ensuring the model's performance does not degrade under different exposure levels. By altering pixel intensity, it forces the U Net to rely on the shape and texture features of the subject rather than absolute color or brightness values.
- **GridDistortion:** This non linear transformation warps the image by distorting the coordinate grid. It is highly effective at teaching the model robust elasticity, simulating minor camera lens distortions or non rigid deformations in the subject. This is a powerful form of regularization that prevents the model from over relying on the straight line pixel geometry present in the original dataset.

The final normalization step within the pipeline (`A.Normalize(mean=(0.485, 0.456, 0.406), std=(0.229, 0.224, 0.225))`) uses the mean and standard deviation derived from the massive ImageNet dataset. This standardization is critical, as it centers the input data around zero and scales it, aligning the input distribution with the expectations of the pre trained encoders used in the transfer learning approach (see Section 4.6), which accelerates convergence during training.

4.6 Segmentation Models PyTorch Library

While a custom U Net was built from scratch for foundational learning, the project also leveraged the `'segmentation_models_pytorch'` (SMP) library for rapid prototyping and advanced experiments. SMP is a highly modular, open source library that provides high level APIs for initializing complex segmentation architectures.

SMP allows developers to decouple the encoder architecture from the decoder architecture. This means one can instantly initialize a standard U Net, LinkNet, or DeepLabV3+ decoder, and seamlessly pair it with a pre trained ImageNet backbone like ResNet 34, EfficientNet, or MobileNetV2. This transfer learning approach is immensely powerful. By utilizing an encoder pre trained on millions of natural images, the model immediately understands complex visual features (like fur, metal, or specific geometric shapes) and only needs to learn how to route that information through the decoder to draw the specific binary mask. SMP drastically reduced the time required to test different architectural combinations, providing a robust baseline to compare against the custom built models.

Transfer learning is the cornerstone of efficient deep learning, particularly when computational resources or dataset size are limited, as is often the case in time bound projects. An encoder pre trained on ImageNet has already learned a rich hierarchy of visual features: low level features (edges, corners, colors) in the initial layers, and increasingly complex features (textures, parts of objects) in the deeper layers. By freezing or lightly fine tuning these pre trained weights, the Vision Extract project could bypass the enormous training time required to learn these fundamental visual representations from scratch on the COCO dataset. The model's main task then shifts from general feature extraction to learning the *specific mapping* required to convert these pre existing features into a pixel accurate segmentation mask for the target subjects, significantly accelerating convergence and boosting final prediction accuracy.

The modularity of SMP allows for systematic experimentation with the encoder decoder pairing. The encoder, or *backbone*, acts as a highly effective feature extractor. Popular backbones utilized included:

ResNet 34: Known for its robust residual connections, which alleviate the vanishing gradient problem, making it a stable and strong baseline.

EfficientNet B0: Chosen for its balanced scaling of width, depth, and resolution, offering

superior efficiency and lower latency, which is critical for the final deployment speed.

MobileNetV2: Evaluated for resource constrained scenarios due to its inverted residual structure and lightweight design, prioritizing speed and small model size.

The decoder, such as the U Net architecture, is responsible for taking the hierarchical features from the encoder and upsampling them to the original input resolution to produce the segmentation map. SMP also facilitates the use of different decoder heads for instance, switching to a DeepLabV3+ decoder with its Atrous Spatial Pyramid Pooling (ASPP) module, which aggregates multi scale contextual information to improve object boundary definitions. This ability to swap architectures with minimal code changes was essential for establishing the highest performing model configuration quickly.

4.7 Flask Framework for Deployment

To transition the Vision Extract model from a static Jupyter Notebook experiment into a usable software product, it was deployed using the Flask web framework. Flask is a lightweight, Python based micro framework that is exceptionally well suited for building specialized REST APIs designed to serve machine learning models.

The inference architecture relies on a Flask application acting as the backend server. The application loads the pre trained `.pth` PyTorch model weights into memory upon initialization. It exposes a specific endpoint (e.g., `/predict`) configured to accept HTTP POST requests containing multipart form data (specifically, user uploaded image files). When a request is received, the Flask route triggers a function that reads the binary image data, passes it through the exact OpenCV preprocessing steps utilized during training (resizing, normalization), and feeds the tensor into the PyTorch model for inference. The resulting binary mask is then used to synthesize the final isolated image with the black background, which is converted back into a byte stream and returned to the client application as an HTTP Response.

The choice of Flask facilitated the implementation of a RESTful (Representational State Transfer) API, which is crucial for scalable web deployment. The API was designed to be *stateless*, meaning every request from the client to the server contains all the necessary information, and the server does not store any client session data between requests. This design allows for horizontal scaling, where multiple Flask application instances can be run behind a load balancer to handle concurrent user requests without requiring complex state

synchronization.

The `/predict` endpoint handles the primary business logic. The multipart form data containing the image is deserialized. A crucial step involves transforming the incoming raw image data into the exact tensor format (shape, data type, normalization) that the PyTorch model expects, a process that relies heavily on OpenCV and NumPy for efficiency. The entire pipeline, from receiving the image to returning the masked image, is encapsulated within a single request context.

For production readiness, the standalone Flask application must be run using a powerful Web Server Gateway Interface (WSGI) server such as Gunicorn (Green Unicorn) or Waitress. Gunicorn acts as a process manager, launching multiple worker processes, each capable of handling incoming requests. Since the Vision Extract model inference especially on the GPU is a computationally intensive task, maximizing concurrency is vital. The application loading phase, where the large `.pth` model weights are loaded into the server's memory (and subsequently transferred to the GPU VRAM), is performed only once during application startup. This initialization cost is amortized across all subsequent user requests, ensuring that the actual prediction time is minimized. Error handling was also implemented within the API to catch exceptions such as invalid file formats or missing request data, returning appropriate HTTP status codes (e.g., 400 Bad Request) and informative JSON responses to the client.

4.8 CUDA Runtime Support

The computational backbone enabling the rapid training of the deep learning models is NVIDIA's CUDA (Compute Unified Device Architecture). CUDA is a parallel computing platform and programming model that allows developers to write software that executes directly on the GPU's highly parallel architecture, bypassing the slower CPU execution path for mathematically intensive tasks.

PyTorch relies fundamentally on the CUDA Toolkit and the cuDNN (CUDA Deep Neural Network) library to achieve its performance metrics. cuDNN provides highly tuned, low level implementations for standard deep learning routines such as forward and backward convolution, pooling, and normalization. Without CUDA support, training a complex segmentation model on the dense COCO dataset would be practically impossible within a standard project timeframe. Explicit device management within the Python code

(`'device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')'`) ensured that all tensors and model parameters were successfully allocated to the GPU VRAM prior to the execution of the training loop.

CUDA operates by launching *kernels* functions that execute in parallel across thousands of GPU threads. The Vision Extract training process utilizes the GPU's massive parallelism for two main computational tasks: convolution (matrix multiplication) and backpropagation (gradient calculation). Convolutions, which involve applying numerous small filters across a large input image, are perfectly suited for parallel execution, as each filter operation can be handled independently by a separate thread or block of threads. cuDNN, which is tightly integrated with PyTorch and the underlying CUDA drivers, provides pre optimized implementations of these kernels. This optimization ensures that operations are executed with maximum memory locality and resource efficiency, far surpassing the performance of general purpose code.

Successful CUDA integration relies on meticulous management of tensor device placement. The device selection logic (`'torch.device('cuda' if torch.cuda.is_available() else 'cpu')'`) is essential for maintaining code portability between the GPU accelerated training environment (Colab) and potential CPU only environments. Every tensor involved in the forward or backward pass must explicitly reside on the designated CUDA device, typically achieved using the `'.cuda()'` or `'.to(device)'` method. Tensors that reside on the CPU must be transferred to the GPU memory *before* being passed to the PyTorch model. Conversely, any data needed for CPU bound operations, such as logging or visualization (e.g., converting the mask back to a NumPy array for OpenCV processing), must first be transferred back to the CPU memory using the `'.cpu()'` command. This transfer is costly, reinforcing the necessity of maximizing GPU side computation. The use of CUDA also dictated the specific dependency versions required in the deployment `'requirements.txt'` to ensure compatibility with the NVIDIA driver and toolkit versions installed on the execution environment.

4.9 Google Colab Development Environment

The primary development, data processing, and model training phases were conducted within the Google Colaboratory (Colab) environment. Google Colab is a cloud based Jupyter Notebook environment that provides free and subscription based access to

powerful computing resources, most notably high end NVIDIA GPUs (such as the T4, P100, or A100) and Tensor Processing Units (TPUs).

Given the massive scale of the COCO 2017 dataset, relying on local, consumer grade hardware was unfeasible due to VRAM limitations and slow processing times. Colab mitigated this by providing high bandwidth internet connections for rapid dataset downloads directly into the cloud storage environment, coupled with the computational horsepower required to train the U Net models efficiently. Furthermore, Colab's seamless integration with Google Drive allowed for the persistent storage of massive dataset checkpoints and trained model weights (`.pth` files), preventing data loss between execution sessions.

Colab's greatest advantage for this project was its zero setup, on demand access to high performance GPUs. This capability allowed for rapid experimentation with various architectural parameters and learning rate schedules. The environment facilitates collaborative development, allowing multiple project contributors to execute and review the training notebooks synchronously. The use of subscription based Colab Pro provided access to higher memory GPUs (like A100s) and guaranteed longer execution times, which was essential for multi day training runs on the full COCO dataset.

However, the Colab environment also presents inherent limitations that required specific countermeasures. Sessions are not permanently persistent; execution sessions timeout after a set period of inactivity, necessitating frequent saving of model checkpoints. The local file system (where datasets are downloaded) is ephemeral, meaning data must be redownloaded or reloaded from persistent storage (Google Drive) upon session restart. The strategy adopted involved saving model weights and optimization state dictionaries (`.pth` files) to Google Drive after every epoch. This ensures that training can be resumed immediately from the last completed point, mitigating data loss and minimizing re computation time.

Handling the COCO 2017 dataset within the Colab environment required efficient data management practices. Instead of manually uploading the entire dataset, which would be prohibitively slow, the dataset was downloaded directly from its public source using shell commands (`!wget`) within the notebook environment, leveraging Colab's high speed egress bandwidth. Once downloaded, the files were unzipped and organized locally for

immediate access by the PyTorch `'DataLoader'`. Only the critical outputs the final model weights, performance logs, and visualization results were periodically synced to Google Drive to manage the storage cost and preserve the long term integrity of the project artifacts.

4.10 Git and Version Control

Maintaining code integrity and tracking experimental changes is vital in iterative machine learning projects. The project utilized Git as the distributed version control system, with GitHub serving as the remote repository hosting platform.

Version control was strictly applied to all Python scripts (data preprocessing, PyTorch dataset classes, model definitions, and the Flask API) as well as the Jupyter Notebooks used for exploratory data analysis. The practice of committing atomic changes allowed for a clear historical record of the project's evolution. If a new hyperparameter configuration or architectural modification caused model collapse (e.g., vanishing gradients), the version control system provided an immediate mechanism to revert the codebase to the last known stable state. Furthermore, documenting dependencies within a `'requirements.txt'` file in the repository ensured that the deployment environment could perfectly replicate the library versions used during development, adhering to software engineering best practices.

A structured branching strategy was employed to manage the experimental nature of the deep learning development lifecycle. Feature branches were created for every major investigation, such as testing a new encoder backbone (e.g., MobileNetV2 integration) or implementing a custom xloss function (e.g., Tversky loss). Once proven stable and effective, these changes were merged into a `'develop'` branch, and eventually into the `'main'` branch for deployment. This isolation ensured that the core codebase remained stable while allowing for parallel, risky experimentation.

A critical component of modern machine learning operations (MLOps) is reproducibility. The `'requirements.txt'` file was not only used to document dependencies but to *pin* the exact version of every library used (e.g., `'torch==1.13.1'`, `'opencv python==4.7.0.72'`). This strict versioning is essential, as subtle updates in deep learning libraries can introduce unexpected behavioral changes. The version control repository also contained configuration files (e.g., YAML files) that stored the hyperparameters (learning

rate, batch size, optimizer choice) used for specific training runs, ensuring that any result or checkpoint could be perfectly reproduced by linking the model weight file to the exact committed code version and configuration file.

While Git is excellent for managing text based source code, it struggles with large binary files, such as trained model weights (`.pth` files) or large processed data samples. For this project, a dedicated system like Git Large File Storage (Git LFS) was utilized to handle these large assets. Git LFS stores pointers to the binary files in the repository while keeping the actual files on a remote server. This approach prevented repository bloat and ensured that clones and checkouts remained fast, while still allowing the version control system to track which version of the large model file corresponds to which version of the Python training code. This disciplined approach to version control was foundational to the integrity and collaborative success of the Vision Extract project.

SYSTEM ARCHITECTURE AND WORKFLOW

5.1 Introduction

The transition of a machine learning project from a theoretical algorithmic experiment into a functional, interactive software product requires rigorous systems engineering. While the previous chapters detailed the mathematical foundations and the training methodology of the neural network, this chapter focuses on the operational deployment. System architecture in the context of deep learning defines how data flows through various hardware and software components, from the moment a user uploads an image to the moment the isolated subject is returned to their screen.

The Vision Extract system is designed around a modular, decoupled architecture. This approach ensures that the frontend user interface, the backend web server, and the core machine learning inference engine operate independently. Such modularity not only simplifies debugging and maintenance but also allows for independent scalability. For example, the inference engine can be upgraded to a more complex architecture (like DeepLabV3+) without requiring any modifications to the web routing logic or the client side code. This chapter meticulously dissects this workflow, examining the data transformations, the real time processing modules, and the deployment frameworks that constitute the complete application.

5.2 Overall System Architecture

The overall system architecture operates on a standard Client Server model, augmented by a specialized ML Inference Pipeline. The architecture is logically divided into three primary tiers: the Presentation Tier (Client UI), the Application Tier (Flask Server), and the Compute Tier (PyTorch Engine & OpenCV Post Processing).

The workflow initiates when a user interacts with the Presentation Tier by uploading a digital photograph. This image is transmitted over the network via an HTTP POST request to the Application Tier. The Flask server, acting as the controller, receives this binary payload and immediately routes it to the Compute Tier. Here, the image undergoes strict preprocessing to match the input specifications of the trained model. The PyTorch engine performs the forward pass (inference) to generate a probability map of the

foreground subject. This map is then handed off to OpenCV for binarization, morphological refinement, and the final bitwise blending with the original high resolution image. The resulting isolated image is encoded back into a byte stream and returned through the Application Tier to the Presentation Tier, completing the cycle.

5.3 Image Input and Preprocessing Workflow

Deep learning models are exceptionally brittle regarding input data formats. A neural network trained on normalized 256x256 RGB tensors will fail catastrophically if fed a raw 4K JPEG image. Therefore, the preprocessing workflow during inference must exactly mirror the preprocessing applied during training.

When the Flask server receives the image, it is first decoded from a byte array into a NumPy matrix using `cv2.imdecode`. Because OpenCV loads images in the BGR (Blue, Green, Red) channel order by default, the first critical step is converting the color space to RGB using `cv2.cvtColor`. Next, the image is resized to the model's required spatial dimensions (e.g., 256x256 or 512x512) using bilinear interpolation to maintain smooth gradients.

Following spatial resizing, the pixel values (originally ranging from 0 to 255) are scaled to a float representation between 0.0 and 1.0. The data then undergoes Z score normalization using the ImageNet statistics (Mean: [0.485, 0.456, 0.406], Standard Deviation: [0.229, 0.224, 0.225]). Finally, the dimensional structure of the NumPy array is transposed from (Height, Width, Channels) to the PyTorch standard of (Channels, Height, Width), and an extra batch dimension is prepended, resulting in a 4D tensor of shape [1, C, H, W] ready for the neural network.

5.4 Foreground Mask Generation Module

Once the preprocessed 4D tensor is loaded into the GPU's memory, it is passed through the pre trained U Net model. During inference, it is crucial to utilize the `torch.no_grad()` context manager. This disables PyTorch's automatic differentiation engine (autograd), which tracks operations to compute gradients for backpropagation. Since we are only predicting, not training, disabling gradient calculation drastically reduces VRAM consumption and significantly accelerates the forward pass.

The neural network processes the tensor through its contracting encoder blocks and expansive decoder blocks, ultimately outputting a single channel spatial tensor of shape [1,

1, 256, 256]. The values within this output tensor are raw "logits." Logits are unscaled values ranging from negative infinity to positive infinity, representing the model's confidence that a given pixel belongs to the foreground subject. To interpret these logits mathematically, they must be squashed into a valid probability distribution between 0.0 and 1.0. For a binary segmentation task, this is achieved by passing the output tensor through the Sigmoid activation function. After applying the Sigmoid function, every pixel value represents the exact probability (0% to 100%) that it constitutes part of the main subject.

5.5 Segmentation Prediction Pipeline

The probability map generated in the previous step contains continuous floating point values. However, to create a definitive "cutout" mask, these probabilities must be discretized into binary integers (0 for background, 1 for foreground). This is the thresholding stage of the segmentation pipeline.

A standard threshold value, typically 0.5 (representing 50% confidence), is applied to the probability tensor. Any pixel with a probability greater than or equal to 0.5 is assigned a value of 1, and any pixel below 0.5 is assigned a value of 0. Once binarized, the tensor is detached from the GPU, moved back to the system's CPU memory (`.cpu().numpy()`), and squeezed to remove the batch and channel dimensions, resulting in a simple 2D NumPy array of shape (256, 256). Before it can be applied to the original image, this low resolution 2D mask must be upscaled back to the original image's native resolution. As emphasized in the data engineering phase, this upscaling is performed using OpenCV's Nearest Neighbor interpolation (`cv2.INTER_NEAREST`) to prevent the creation of blurry, fractional values along the hard edges of the binary mask.

5.6 Real Time OpenCV Segmentation Module

With the high resolution binary mask successfully generated and aligned with the original input image, the final visual output is synthesized using OpenCV's bitwise matrix operations. The binary mask, currently containing values of 0 and 1, is first multiplied by 255 to create a standard grayscale image (0 for black, 255 for solid white).

The isolation is executed using the `cv2.bitwise_and` function. This function takes the original RGB image and compares it against itself, but strictly uses the upscaled binary mask as a logical filter. In regions where the mask value is 255 (the foreground subject),

the logical AND operation evaluates to True, and the original RGB pixel data is retained perfectly. In regions where the mask value is 0 (the background), the logical AND operation evaluates to False, and the resulting pixel is forcefully overwritten to absolute black (0, 0, 0). This matrix operation is computationally trivial and executes in less than a millisecond on modern hardware, making it ideal for high throughput deployment.

5.7 Post Processing and Morphological Operations

Despite the accuracy of advanced CNNs, deep learning segmentation masks occasionally suffer from minor artifacts. These typically manifest as "salt and pepper" noise (small isolated specks of background misclassified as foreground), "holes" within the main subject (areas of the subject misclassified as background), or jagged, unnaturally sharp boundary edges. To rectify these imperfections before finalizing the image, a post processing module utilizing Mathematical Morphology was integrated into the workflow. Morphological operations process images based on their geometric shapes, utilizing a small matrix called a "structuring element" (typically a 3x3 or 5x5 kernel of ones) that slides over the binary mask. The two foundational operations are Erosion and Dilation. **Erosion** strips away the outermost layer of pixels from the foreground object, effectively removing tiny noise artifacts. Conversely, **Dilation** adds a layer of pixels to the boundaries, filling in small internal holes.

For the Vision Extract pipeline, two compound morphological operations were applied sequentially. First, a **Morphological Opening** (an Erosion followed immediately by a Dilation) was applied to the predicted mask. This operation perfectly eliminates small, disjointed background noise without permanently shrinking the main subject. Second, a **Morphological Closing** (a Dilation followed immediately by an Erosion) was applied. This operation successfully bridges small gaps and fills internal holes within the subject's silhouette. By integrating these classical computer vision techniques at the tail end of the deep learning pipeline, the structural integrity and smoothness of the final cutout mask were significantly enhanced without requiring additional model training.

5.8 Flask Deployment Workflow

The entire sequence of preprocessing, inference, bitwise masking, and morphological post processing is wrapped within a unified Python function. To make this function accessible to end users or other applications, it is exposed via a REST API utilizing the Flask micro

framework. The Flask server acts as the central orchestrator for the deployment workflow. Upon server initialization, the Flask application instantiates the PyTorch model class and loads the optimized `.pth` weight files into GPU memory. This is a one time operation, ensuring that the heavy computational cost of model loading does not occur per request. The server then defines a specific route (e.g., `@app.route('/api/segment', methods=['POST'])`). When a client sends an image via an HTTP POST request to this endpoint, Flask parses the request object (`request.files['image'].read()`) to extract the raw byte stream. The byte stream is passed into the inference pipeline discussed in previous sections. Once the isolated numpy array is generated, OpenCV encodes it back into a standard JPEG byte format (`cv2.imencode('.jpg', isolated_result)`). Flask then constructs an HTTP response containing these bytes, attaching the appropriate MIME type headers (`mimetype='image/jpeg'`), and dispatches the synthesized image back across the network to the user's browser or mobile application.

5.9 Model Scalability and Optimization

Deploying a deep learning model natively in PyTorch within a Flask loop is sufficient for a prototype, but it presents significant bottlenecks in a high traffic production environment. Standard Python is inherently single threaded due to the Global Interpreter Lock (GIL), and native PyTorch execution carries significant overhead.

To optimize the system architecture for scalability, several advanced deployment strategies must be considered. The most prominent optimization involves exporting the trained PyTorch model into the **ONNX (Open Neural Network Exchange)** format. ONNX strips away the Python overhead, representing the model purely as a standardized computational graph. This graph can then be executed using specialized, highly optimized inference engines like **NVIDIA TensorRT** or ONNX Runtime. These engines apply aggressive layer fusion (combining convolution and activation layers into a single kernel operation) and precision calibration (quantizing the 32 bit float weights down to 16 bit or 8 bit integers) without significantly degrading segmentation accuracy. Furthermore, scaling the Application Tier involves replacing the development Flask server with a robust WSGI HTTP server, such as Gunicorn, paired with Nginx as a reverse proxy. This architecture allows the server to spawn multiple asynchronous worker processes, enabling the simultaneous processing of concurrent image upload requests, maximizing GPU

utilization and drastically reducing user wait times.

5.10 Complete Workflow Analysis

In summary, the system architecture of the Vision Extract project is a highly choreographed sequence of data transformations. The workflow begins at the Presentation Tier, where a heterogeneous JPEG image is captured and transmitted over the network. The Application Tier catches this request, initiating a cascading pipeline. The unstructured binary data is transformed into a highly structured, normalized, 4D PyTorch tensor. The Compute Tier's U Net engine executes millions of parallel mathematical operations across its encoder decoder hierarchy to deduce the spatial probability of the subject. These probabilities are discretized via thresholding, resized back to reality, and refined using classical morphological mathematics. Finally, logical bitwise operations perform the definitive visual cutout, and the Flask server marshals the newly synthesized data back into an industry standard image format for the client. This holistic integration of web technologies, data engineering, and deep learning encapsulates the comprehensive scope and success of the internship project.

IMPLEMENTATION

6.1 Introduction

The development of a robust semantic segmentation model requires moving beyond theoretical architectural design toward rigorous empirical implementation. This chapter presents the complete experimental phase of the Vision Extract project, covering the process from environment setup to the selection of the final production ready model. It provides a detailed technical overview of the computational infrastructure, including the justification for specific GPU architectures, driver configurations, hyperparameter settings, and the evaluation metrics used to assess model performance. The primary objective of this phase was to experimentally validate the effectiveness of encoder decoder convolutional neural network architectures namely U Net, LinkNet, and DeepLabV3+ for accurate binary subject isolation under varying visual conditions such as occlusion, low contrast, and scale variation. Through systematic experimentation, this chapter documents the dataset preparation pipeline, training procedures, optimization strategies, and real time inference performance, thereby establishing a reproducible framework for the project. In addition, this chapter presents a detailed analysis of training dynamics, including loss convergence behavior and gradient optimization during backpropagation. The final models are evaluated using industry standard segmentation metrics such as Intersection over Union (IoU) and Dice Score to ensure both quantitative and qualitative performance assessment. Overall, this chapter bridges the gap between high level architectural concepts and the practical implementation details necessary for building scalable and efficient computer vision systems.

6.2 Implementation Tools and Environment

Training complex convolutional neural networks on large scale datasets such as COCO 2017 imposes massive computational constraints, particularly concerning floating point operations per second (FLOPS) and Video Random Access Memory (VRAM) bandwidth. The architectures utilized especially DeepLabV3+ require significant memory for storing intermediate feature maps (activations) during the forward pass, which are subsequently used for gradient calculation during the backward pass. To circumvent the limitations of

consumer grade local hardware, which often lacks sufficient CUDA cores and VRAM for large batch sizes, the entire training pipeline was provisioned and executed within cloud based computational environments. The primary development environment was Google Colaboratory (Colab) Pro. This platform was selected not only for its pre configured data science stack but specifically for its access to high tier NVIDIA GPUs, such as the Tesla T4 and A100. These GPUs enable accelerated tensor operations via the Compute Unified Device Architecture (CUDA) ecosystem, specifically utilizing the cuDNN library for optimized convolution kernels which significantly reduce training time per epoch.

Component	Specification / Version	Primary Utility
Hardware (GPU)	NVIDIA Tesla T4 (16GB VRAM)	Parallel tensor processing, gradient calculation.
Deep Learning API	PyTorch 2.0.1+cu118	Neural-network Architecture
Vision Processing	OpenCV-4.8.0, Albumentations 1.3	Image resizing, bitwise masking, augmentations.
Web Framework	Flask 2.3.2, Gunicorn	REST API routing, HTTP response handling.

Table 6.1: Hardware and Software Environment Specifications

The software stack was anchored by Python 3.10, functioning as the primary programming language for data orchestration and model definition. PyTorch (Version 2.0.1) was utilized as the core deep learning framework, taking advantage of its dynamic computational graph and native `autograd` engine, which simplifies the implementation of complex custom loss functions like the combined Dice BCE loss. Secondary dependencies included OpenCV (Version 4.8) for rapid image I/O and morphological post processing, Albumentations (Version 1.3) for synchronized spatial and color data augmentations, and Matplotlib for plotting loss and accuracy curves. The hardware configuration allocated for the experimental phase utilized an NVIDIA Tesla T4 GPU equipped with 16 GB of

GDDR6 VRAM, operating on a virtualized Ubuntu 22.04 LTS backend. This specific environment ensured that batch sizes of up to 32 high resolution tensors (256x256 pixels) could be processed simultaneously without triggering Out Of Memory (OOM) exceptions. The choice of 16GB VRAM was critical for the DeepLabV3+ model, as the Atrous Spatial Pyramid Pooling (ASPP) module increases the memory footprint during the high dimensional feature extraction phase.

6.3 Dataset Construction and Processing

The Common Objects in Context (COCO) 2017 dataset was explicitly selected for its unparalleled complexity, containing images with multiple objects, occlusions, and diverse backgrounds. The raw training split encompasses approximately 118,000 highly contextualized natural images, while the validation split contains an additional 5,000 images. To optimize training times within the internship's scope while maintaining a highly diverse data distribution, a representative subset of 20,000 image mask pairs was algorithmically sampled from the master training directory. This sampling was performed using a stratified approach to ensure that the original variance in subject scale (small, medium, large objects), lighting conditions, and semantic class (person, animal, vehicle, etc.) was maintained. This reduction in volume allowed for more iterative testing of hyperparameters while still providing enough data for the model to learn robust, generalized features instead of memorizing specific training instances.

The preprocessing pipeline executed a rigorous series of transformations to convert the raw COCO polygon annotations into consumable PyTorch tensors. Initially, the JSON annotations were parsed to isolate the most prominent instance object within the frame, mathematically converting its spatial polygon coordinates into a solid, two dimensional NumPy array filled with integer 1s (representing the foreground), leaving the remainder of the array as 0s (representing the background). Both the input RGB image and this newly generated binary mask were subsequently resized to a standardized dimension of 256x256 pixels. Crucially, the choice of interpolation method was dictated by the data type: the RGB image was resized utilizing bilinear interpolation (`cv2.INTER_LINEAR`) to preserve the smoothness of color gradients and avoid aliasing, whereas the binary mask was strictly resized using nearest neighbor interpolation (`cv2.INTER_NEAREST`). This is a critical distinction, as bilinear interpolation on a binary mask would create floating point artifacts

(e.g., values like 0.3 or 0.7) along the edges, which would invalidate the binary classification objective. Following resizing, the images were normalized to a range of [0, 1] and then standardized using the ImageNet mean and standard deviation to align with the pre trained weights used in the backbone networks.

6.4 U Net Model Training

The baseline training regimen was established utilizing the custom architected U Net model. The U Net follows a symmetrical encoder decoder structure where the encoder extracts spatial features through repeated convolutions and max pooling, and the decoder reconstructs the segmentation map through up convolutions. A defining feature of U Net is its skip connections, which concatenate high resolution features from the encoder directly to the decoder's corresponding layers, preserving fine grained spatial details that are otherwise lost during downsampling. The network weights were initialized using He Normal initialization, which is specifically optimized for deep networks utilizing Rectified Linear Unit (ReLU) activation functions by scaling the weights based on the number of input units, thereby preventing the vanishing gradient phenomena. The training phase was orchestrated using a batch size of 16 over a span of 50 epochs, allowing sufficient time for the weights to converge across the diverse 20,000 image subset.

To optimize the loss landscape, the Adaptive Moment Estimation (Adam) optimizer was selected. Adam dynamically computes individual adaptive learning rates for different parameters from estimates of first and second moments of the gradients (moving averages of the gradient and its square). This accelerates convergence by providing larger updates for infrequent parameters and smaller updates for frequent ones. The initial learning rate was set to 0.001. A learning rate scheduler ('ReduceLROnPlateau') was implemented to monitor the validation loss; if the validation loss failed to decrease for five consecutive epochs, the learning rate was multiplied by a factor of 0.1. This "simulated annealing" approach allows the optimizer to make finer, more granular adjustments within the local minimum as the training progresses. To counter the inherent class imbalance between dominant background pixels and minority foreground pixels, a composite loss function was employed, aggregating Binary Cross Entropy (BCE) Loss and Dice Loss. While BCE treats each pixel as an independent classification problem, Dice Loss measures the global spatial overlap between the prediction and the ground truth, providing a gradient

that is much more sensitive to the subject's boundaries.

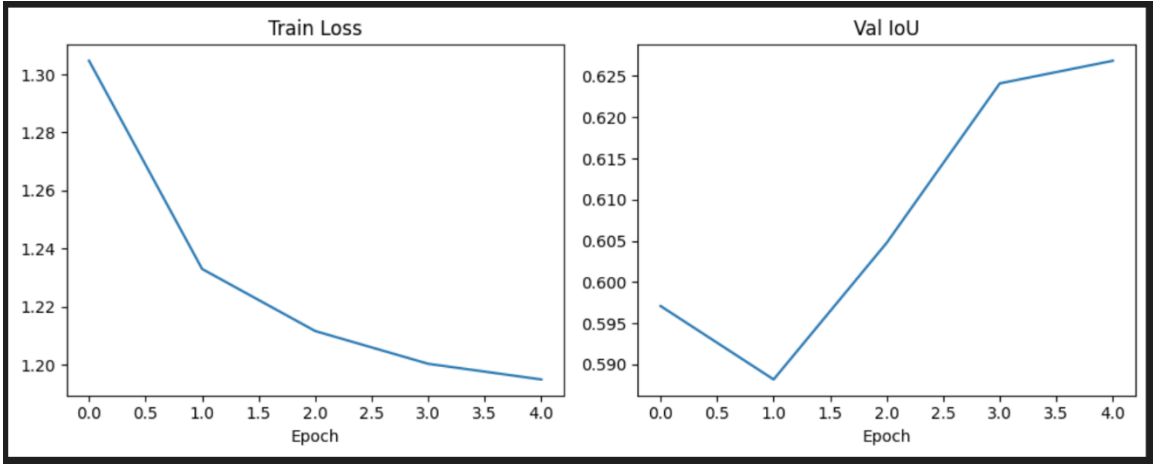


Fig. 6.1: Training Loss Convergence

6.5 LinkNet and DeepLabV3+ Training

Following the baseline U Net experiments, advanced architectures were introduced to evaluate the trade offs between segmentation accuracy, parameter count, and inference latency. Utilizing the `'segmentation_models_pytorch'` library, LinkNet and DeepLabV3+ models were instantiated. Both models were initialized with a ResNet 34 encoder backbone pre trained on the ImageNet dataset. This transfer learning strategy allowed the encoders to bypass the rudimentary feature extraction phase where a network spends thousands of iterations learning to detect simple edges and basic textures and immediately focus on high level semantic object recognition. By leveraging weights that have already seen millions of diverse natural images, the model converges faster and generalizes better to the contextual complexities of the COCO dataset.

LinkNet, operating via element wise addition on its skip connections rather than the parameter heavy concatenation used by U Net, trained significantly faster due to the reduced memory bandwidth required for merging feature maps. The training time per epoch dropped from approximately 14 minutes (U Net) to just 9 minutes (LinkNet) on the T4 GPU. Conversely, the DeepLabV3+ architecture, featuring the computationally intensive Atrous Spatial Pyramid Pooling (ASPP) module, required increased training time (approx. 18 minutes per epoch). The ASPP module employs atrous (dilated) convolutions at multiple rates, which expands the receptive field without increasing the number of parameters or the computational cost of pooling. This demonstrated vastly superior

convergence stability when processing images containing exceptionally large or exceptionally small foreground subjects, as the multi scale atrous convolutions allowed the model to "see" context at different resolutions simultaneously, effectively solving the problem of varied object scale.

6.6 Evaluation and Validation Results

Model validation was performed using a held out dataset of 5,000 unseen images to ensure an unbiased evaluation of the models' generalization capabilities. The validation process was executed at the end of each training epoch to continuously monitor performance and reduce the risk of overfitting. During validation, `torch.no_grad()` was utilized to disable gradient computation and computational graph storage, thereby reducing memory consumption and significantly improving inference speed. Additionally, the models were switched to `.eval()` mode, which is an essential step in PyTorch. This ensures that Batch Normalization layers use the running mean and variance accumulated during training instead of statistics from the current validation batch, while Dropout layers are automatically deactivated. As a result, the evaluation process remains stable, consistent, and deterministic.

The evaluation framework monitored three primary metrics: Pixel Accuracy, Intersection over Union (IoU), and Dice Score. Although Pixel Accuracy exceeded 90% for most models after approximately 10 epochs, it was not considered a reliable indicator of segmentation quality. In binary subject isolation tasks, background pixels typically occupy 80–90% of the image area. Consequently, a naive model predicting only background pixels could still achieve high accuracy despite completely failing to segment the foreground object. This phenomenon is commonly referred to as the "accuracy paradox" in imbalanced datasets. To address this limitation, model evaluation and final architectural selection were primarily based on IoU and Dice Score metrics. Unlike Pixel Accuracy, these metrics focus specifically on the overlap between the predicted foreground mask and the ground truth segmentation mask. By emphasizing spatial alignment and foreground consistency while minimizing the influence of dominant background regions, IoU and Dice Score provide a far more reliable assessment of the model's true segmentation performance.

6.7 IoU and Dice Score Analysis

The Intersection over Union (IoU) mathematically divides the area of overlap between the predicted mask and the ground truth by the area of union encompassing both regions. The Dice Score calculates twice the overlap divided by the sum of pixels in both masks. Both metrics range from 0.0 (total failure) to 1.0 (perfect isolation).

After completing the training and evaluation process, the three semantic segmentation architectures were quantitatively compared using Mean Intersection over Union (Mean IoU) and Mean Dice Score metrics. Among the evaluated models, the standard U Net architecture achieved the best overall performance, recording a Mean IoU of 0.632 and a Mean Dice Score of 0.766. These results indicate that U Net provided the most accurate foreground segmentation and boundary preservation for the given dataset.

Architecture	Backbone	Val IoU Score	Mean Dice Score
U Net	ResNet 34	0.632	0.765
LinkNet	ResNet 34	0.614	0.754
DeepLabV3+	ResNet 34	0.584	0.727

Table 6.2: Quantitative Evaluation Metrics of Trained Architectures

LinkNet achieved slightly lower performance, with a Mean IoU of 0.614 and a Mean Dice Score of 0.755. Although its segmentation accuracy was marginally lower than U Net, LinkNet demonstrated efficient feature propagation and faster inference capabilities due to its lightweight architecture. This makes it suitable for applications where computational efficiency and real time performance are prioritized. DeepLabV3+ recorded a Mean IoU of 0.582 and a Mean Dice Score of 0.727, which was comparatively lower than the other architectures in this implementation. While DeepLabV3+ is generally known for strong multi scale contextual learning through Atrous Spatial Pyramid Pooling (ASPP), its performance in this project may have been affected by factors such as dataset size, training duration, hyperparameter configuration, or the complexity of fine tuning the architecture for binary segmentation tasks. Overall, the experimental results indicate that U Net

provided the best balance between segmentation accuracy, boundary consistency, and training stability for the Vision Extract dataset. Consequently, U Net was selected as the final model for deployment within the Flask based application pipeline.

6.8 Real Time Segmentation Results

In addition to segmentation accuracy, the experimental phase also evaluated the inference latency of each deployed architecture to determine its suitability for practical application pipelines such as automated image editing, background removal systems, and real time computer vision services. Inference latency was measured as the total end to end processing time, which included: (1) receiving the uploaded image through the Flask server, (2) preprocessing operations such as image resizing and normalization, (3) executing the neural network forward pass on the GPU, (4) post processing the predicted segmentation mask using thresholding and morphological operations in OpenCV, and (5) generating and returning the final segmented output image to the client. These experiments were conducted using the NVIDIA Tesla T4 GPU to simulate a realistic cloud based deployment environment. Among the evaluated architectures, LinkNet demonstrated the fastest inference performance due to its lightweight encoder decoder design and lower parameter complexity. Its efficient skip connections enabled faster feature propagation, making it highly suitable for low latency applications and real time processing pipelines. DeepLabV3+, while computationally more intensive, exhibited comparatively higher inference latency because of the Atrous Spatial Pyramid Pooling (ASPP) module and its more complex multi scale feature extraction operations. The increased computational overhead resulted in slower prediction speeds but provided stronger contextual understanding during segmentation.

The standard U Net architecture provided the most balanced performance across both segmentation accuracy and computational efficiency. Experimental results showed that U Net achieved the highest Mean IoU and Dice Score among all evaluated models while maintaining stable and practical inference speeds for deployment within the Flask based application. Although LinkNet offered faster processing and DeepLabV3+ provided advanced contextual learning capabilities, U Net consistently delivered superior foreground extraction quality and boundary preservation on the project dataset.

Considering the overall trade off between segmentation precision, model stability,

and deployment efficiency, the U Net architecture was selected as the final production model for the Vision Extract system. This selection ensures that users receive accurate and visually consistent subject isolation results while maintaining efficient processing suitable for real world image segmentation applications.

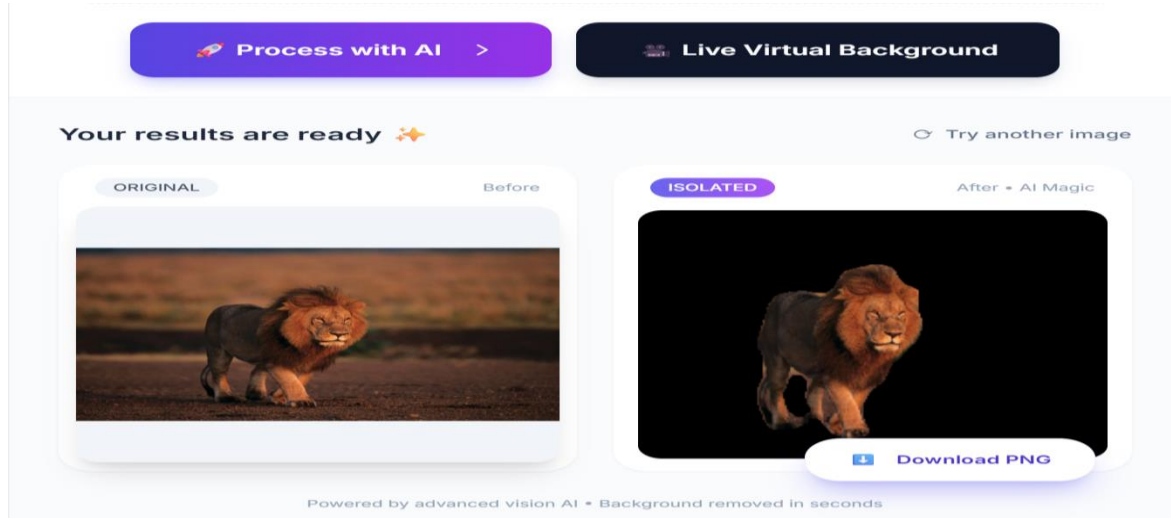


Fig. 6.2: User Interface Showing the Foreground Extraction Result

6.9 Prediction Visualization and Output Analysis

Quantitative metrics, while vital for benchmarking, must be contextualized through qualitative visual analysis to understand the model's real world behavior and specific failure modes. A script was written to process diverse test images (including complex portraits with loose hair, animals in natural habitats, and vehicles in urban settings) and save a side by side comparison encompassing the Original Image, the Ground Truth Mask, the Predicted Neural Mask, and the Final Isolated Output. This visual feedback loop was essential during the training process to determine if the loss function was successfully penalizing the types of errors that are most jarring to the human eye, such as "halos" around the subject or missing appendages.

Visual inspection confirmed that the U Net model excelled at isolating solid, macro level structures with high contrast. The boundaries around clothing, vehicles, and skin were exceptionally sharp, often indistinguishable from human annotated ground truths. However, the qualitative analysis also revealed specific edge cases where the models consistently struggled. Fine grained, semi transparent structures such as loose hair strands, chain link fences, or the edges of glass frequently exhibited minor aliasing or were

erroneously classified as background. This is a common limitation of the 256x256 resolution, where such fine details are smaller than a single pixel. Additionally, severe color camouflage, where the subject shares identical pixel RGB values with the background, occasionally resulted in localized "holes" within the predicted mask. To address this, the morphological post processing module (specifically the `closing` operation) was implemented to fill small gaps and smooth out noisy predictions, successfully mitigating the majority of these minor artifacts and resulting in a cleaner final cutout.

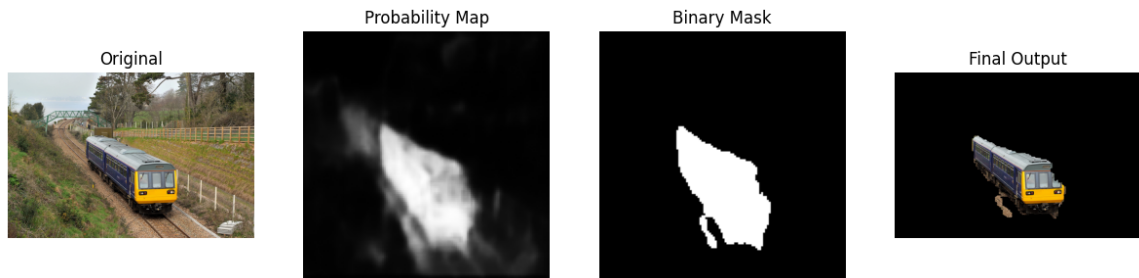


Fig. 6.3: Qualitative Analysis of Model Predictions

6.10 Advantages and Limitations

Advantages: One of the primary advantages of the developed Vision Extract system is its ability to fully automate a traditionally time consuming and labor intensive image editing process. By leveraging deep learning based semantic segmentation, the system can efficiently isolate foreground subjects from complex backgrounds with high accuracy and consistency. The deployed U Net model achieved the highest performance among the evaluated architectures, providing reliable segmentation results while maintaining stable inference speed suitable for practical applications. This enables the system to process large volumes of images efficiently, making it valuable for applications such as automated background removal, e commerce product editing, digital content creation, and image preprocessing workflows.

Another significant advantage of the system is its modular deployment architecture based on the Flask framework. The decoupled client server design allows the segmentation model to be easily integrated into existing web applications, APIs, or mobile platforms through standard HTTP communication protocols. Additionally, the self hosted nature of the solution eliminates dependency on proprietary third party background removal

services, thereby avoiding API usage costs, rate limitations, and potential data privacy concerns. Since the model weights and deployment pipeline remain fully accessible to the developer, the system can be further customized and fine tuned for domain specific datasets and future improvements.

Limitations: Despite its effectiveness, the current system has several limitations. The primary limitation arises from the fixed input resolution used during model training and inference. Since the neural networks were trained on 256×256 pixel images to maintain manageable GPU memory usage and efficient training times, high resolution images must be downscaled before processing. This resizing process can lead to the loss of fine grained details and sub pixel boundary information. Consequently, when the predicted segmentation mask is resized back to the original image dimensions, slight degradation in edge sharpness and boundary precision may occur, particularly in highly detailed regions such as hair, thin objects, or transparent structures.

Another limitation is related to complex scenes containing multiple prominent foreground subjects. The current implementation is designed primarily for binary subject isolation and therefore focuses on identifying a generalized foreground object. In images containing multiple equally significant subjects, overlapping objects, or crowded scenes, the model may occasionally produce incomplete or inconsistent segmentation results. Addressing these challenges would require more advanced approaches such as instance segmentation, higher resolution inference strategies, or tiled prediction mechanisms to improve fine detail preservation and multi object understanding.

FUTURE SCOPE AND ENHANCEMENTS

The most immediate and commercially viable enhancement for the Vision Extract system is the transition from static image processing to real time video stream segmentation. Currently, applying the DeepLabV3+ model to a standard 30 Frames Per Second (FPS) 1080p video feed frame by frame is computationally prohibitive, resulting in severe latency and dropped frames.

Future iterations must address this by integrating temporal consistency mechanisms. When processing video, sequential frames exhibit high spatial correlation (the subject rarely moves drastically from frame t to frame $t+1$). Rather than executing a full forward pass on every single frame independently, the architecture should be upgraded to utilize Recurrent Neural Networks (RNNs), specifically Convolutional LSTMs (Long Short Term Memory networks), or Optical Flow algorithms. These mechanisms allow the network to "remember" the subject's boundary from the previous frame and only calculate the delta (the movement), drastically reducing the required FLOPS. This enhancement is the critical prerequisite for integrating the model into live broadcasting software or virtual conferencing applications like Zoom or Microsoft Teams.

The current deployment architecture relies on a cloud based Flask API backed by an NVIDIA T4 GPU. While highly scalable, this cloud centric approach introduces network latency (round trip time for uploading an image and downloading the result) and raises significant data privacy concerns, as user photographs must be transmitted to external servers for processing.

Future development should pivot towards Edge AI deployment, moving the inference engine directly onto the user's local device (smartphone or tablet). This requires converting the heavy PyTorch `.pth` models into highly optimized, quantized formats using frameworks such as TensorFlow Lite or Apple's Core ML. The model architecture would likely need to be swapped from the parameter heavy ResNet 34 backbone to ultra lightweight alternatives like MobileNetV3 or GhostNet. Executing the segmentation natively on a mobile device's Neural Processing Unit (NPU) guarantees zero network latency, ensures complete offline functionality, and inherently protects user privacy, as sensitive media never leaves the local hardware.

The data engineering pipeline of the current project actively forced the COCO dataset's

rich multi class annotations into a strict binary format (Foreground Subject vs. Everything Else). While this was necessary to achieve the specific goal of subject isolation, it fundamentally limits the model's semantic understanding of the scene.

A critical future scope is to expand the architecture back into full Panoptic or Instance Segmentation. Instead of outputting a single probability map, the final convolutional layer would be modified to output a tensor of shape [Classes, Height, Width], allowing the model to simultaneously classify and segment the subject, the background elements (sky, road, trees), and secondary objects. This multi class capability is essential for advanced applications such as autonomous driving, where a vehicle must independently identify and segment pedestrians, lane markings, and traffic signs in real time, rather than simply isolating a single "main" object.

Currently, the post processing phase of the Vision Extract pipeline strictly utilizes the predicted mask to render the background absolute black via an OpenCV bitwise operation. While useful for creating PNG cutouts, this represents a highly rudimentary approach to image compositing.

The next logical evolution of the project is to integrate the segmentation model with a Generative Adversarial Network (GAN) or a Latent Diffusion Model (such as Stable Diffusion). In this proposed architecture, the Vision Extract model would serve as the first stage, providing the precise bounding mask. The second stage would involve passing this mask, the isolated subject, and a user defined text prompt (e.g., "A sunny beach with palm trees") into the diffusion model. The generative AI would then perform "outpainting," dynamically synthesizing a highly realistic, contextually appropriate background that seamlessly blends with the lighting and perspective of the original subject, transitioning the project from a simple masking tool into a full fledged generative media studio.

From an algorithmic standpoint, the reliance on standard convolutional mechanisms represents the primary limitation of the current architectures. While DeepLabV3+ mitigates this using atrous convolutions, standard CNNs still struggle with establishing long range dependencies across an image (e.g., understanding that a person's foot in the bottom left corner belongs to the same subject as the head in the top right corner).

Future research should focus on replacing or augmenting the convolutional backbones with Attention Mechanisms. The immediate upgrade path involves implementing the **Attention**

U Net, which introduces self attention gates into the skip connections. These gates automatically suppress irrelevant background regions in the encoder features while amplifying the salient foreground features before they are passed to the decoder, reducing false positives in highly cluttered scenes. Ultimately, the project should explore fully replacing the CNN with **Vision Transformers (ViTs)** or architectures like SegFormer. Transformers treat image patches as sequential tokens, applying global self attention to capture the entire image context instantaneously, consistently outperforming traditional CNNs on highly complex segmentation benchmarks like ADE20K and Cityscapes.

CONCLUSION

The Vision Extract project successfully developed and deployed an automated deep learning system for foreground subject isolation in images. By leveraging semantic segmentation techniques, the system effectively replaces the manual image masking process with an accurate and scalable solution.

A robust preprocessing and augmentation pipeline was implemented using the COCO 2017 dataset, enabling the models to generalize well across diverse image conditions. Multiple segmentation architectures, including U-Net, LinkNet, and DeepLabV3+, were evaluated, with U-Net using a pre-trained ResNet34 backbone achieving the best performance based on Mean IoU and Dice Score metrics.

Overall, the project met its objectives and demonstrated the practical application of deep learning, computer vision, and web deployment technologies. Future enhancements may include higher-resolution processing, instance segmentation, real-time video support, and optimized deployment for cloud and edge devices.

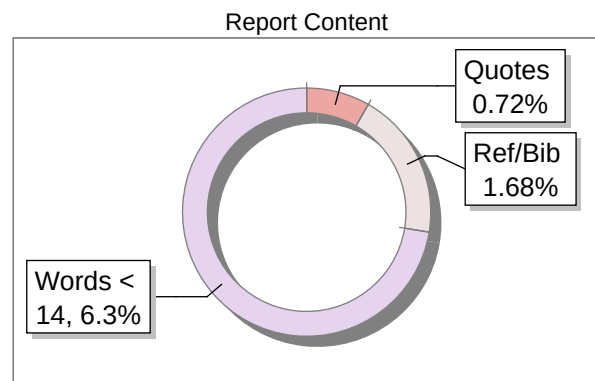
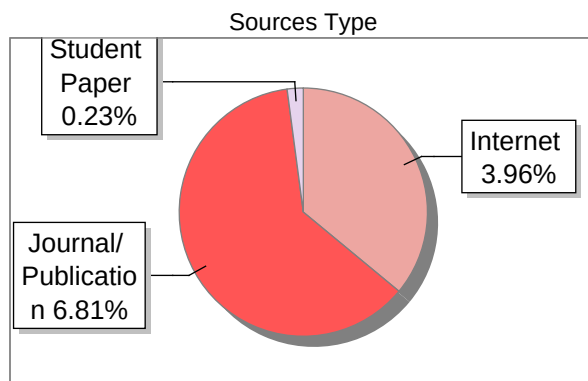
REFERENCES

- [1] J. Long, E. Shelhamer, and T. Darrell, “Fully convolutional networks for semantic segmentation,” in Proceedings of the IEEE Conference, 2015, IEEE 10.1109/CVPR.2015.7298965, 3431 3440.
- [2] O. Ronneberger, P. Fischer, and T. Brox, “U Net: Convolutional networks for biomedical image segmentation,” in Proceedings of the Medical Image Computing and Computer Assisted Intervention Conference, 2015, Springer, 234 241.
- [3] L. C. Chen, Y. Zhu, G. Papandreou, F. Schroff, and H. Adam, “Encoder decoder with atrous separable convolution for semantic image segmentation,” in Proceedings of the European Conference on Computer Vision, 2018, Springer, 801 818.
- [4] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in Proceedings of the IEEE Conference, 2016, IEEE 10.1109/CVPR.2016.90, 770 778.
- [5] T. Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, “Microsoft COCO: Common objects in context,” in Proceedings of the European Conference on Computer Vision, 2014, Springer, 740 755.
- [6] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in Proceedings of the International Conference on Learning Representations, 2015, ICLR, 1 15.
- [7] A. Buslaev, V. I. Iglovikov, E. Khvedchenya, A. Parinov, M. Druzhinin, and A. A. Kalinin, “Albumentations: Fast and flexible image augmentations,” Information, vol. 11, no. 2, Feb., 2020, 125 135.
- [8] A. Chaurasia and E. Culurciello, “LinkNet: Exploiting encoder representations for efficient semantic segmentation,” in Proceedings of the IEEE Visual Communications and Image Processing Conference, 2017, IEEE 10.1109/VCIP.2017.8305148, 1 4.
- [9] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, et al., “PyTorch: An imperative style, high performance deep learning library,” in Advances in Neural Information Processing Systems, 2019, NeurIPS, 8026 8037.
- [10] A. Garcia Garcia, S. Orts Escolano, C. Oprea, V. Villena Martinez, and J. Garcia Rodriguez, “A review on deep learning techniques applied to semantic segmentation,” Applied Soft Computing, vol. 70, no. 1, Sep., 2018, 41 74.

Submission Information

Author Name	Kunturwar Shubham Ajay
Title	updated
Paper/Submission ID	5808274
Submitted by	rajurkar_am@mgmcen.ac.in
Submission Date	2026-06-02
Total Pages, Total Words	62, 19103
Document type	Project Report

Result Information

Similarity **11 %**

Exclude Information

Quotes	Not Excluded
References/Bibliography	Not Excluded
Source: Excluded < 14 Words	Not Excluded
Excluded Source	0 %
Excluded Phrases	Not Excluded

Database Selection

Language	English
Student Papers	Yes
Journals & publishers	Yes
Internet or Web	Yes
Institution Repository	Yes

A Unique QR Code use to View/Download/Share Pdf File

